Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп’ютерних систем
Кафедра інформаційних систем

Нікітін Павло Якович,
студент групи ЛАІ-195

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

Розробка мобільного застосунку для збору та агрегації даних про вакансії

Спеціальність:
122 Комп’ютерні науки

Освітня програма:
Комп’ютерні науки

Керівник:
Петросюк Денис Валерійович,
асистент

Одеса – 2023

ЗМІСТ

ЗАВДАННЯ 5

АНОТАЦІЯ 7

ВСТУП 8

1 ОГЛЯД ІСНУЮЧИХ СИСТЕМ ЗБОРУ ТА АГРЕГАЦІЇ ДАНИХ ПРО ВАКАНСІЇ 9

1.1 Порівняння аналогів систем збору та агрегації даних про вакансії. 9

1.1.1 Jobs.ua 9

1.1.2 Work.ua 12

1.1.3 Djinni 16

1.1.4 Dou.ua 20

1.1.5 Результат порівняння 22

1.2 Вибір технологій для розробки системи 23

1.2.1 Вибір мови програмування 23

1.2.2 Вибір середовища програмування 25

1.2.3 Вибір фреймворків та бібліотек 26

1.2.4 Вибір системи управління базами даних 28

1.2.5 Вибір патернів проектування 29

1.3 Висновки до першого розділу 30

2 ПРОЕКТУВАННЯ ЗАСТОСУНКУ ЗБОРУ ТА АГРЕГАЦІЇ ДАНИХ ПРО ВАКАНСІЇ 32

2.1 Мета та задачі ІС 32

2.2 Типи користувачів 34

2.3 Функціональні вимоги 34

2.4 Моделювання прецедентів (Use case View) 36

2.5 Нефункціональні вимоги 37

2.6 Ідентифікація архетипу ІС 38

2.7 Користувацький інтерфейс (UI View) 38

2.8 Логічне уявлення про ІС (Logical View) 43

2.9 Уявлення розгортання ІС (Deployment View) 44

2.10 Уявлення шарів ІС (Design View) 45

2.11 Уявлення процесів ІС (Process View) 47

2.12 Проектування бази даних 48

2.13 Уявлення інтерфейсів ІС (UI View) 49

2.14 Уявлення безпеки ІС 50

2.15 Опис стеку технологій 51

2.16 Висновок до другого розділу 51

3 РЕАЛІЗАЦІЯ ІНФОРМАЦІЙНОЇ ПІДТРИМКИ ТУРИСТИЧНИХ ПОДОРОЖЕЙ З ПЛАНУВАННЯМ МІСЦЬ ВІДПОЧИНКУ 53

3.1 Уявлення про структуру проекту ІС 53

3.2 Уявлення про структуру класів 53

3.3 Інфраструктурне уявлення ІС (Infrastructure View) 53

3.4 Стратегія доставки продукту (Delivery Strategy View) 53

3.5 Управління програмним кодом системи 53

3.6 Розрахунок метрики програмного коду системи 53

3.7 Контрольний список по якості реалізації ІС 53

3.8 Тестування інформаційної системи 53

3.8.1 Розробка тест-кейсів для функціонального тестування системи 53

3.8.2 Протокол проведення функціонального тестування системи (Functional acceptance testing) 53

3.9 Інструкція користувача 53

3.10 Висновки до третього розділу 53

ВИСНОВКИ 54

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ 55

ДОДАТОК А ЛІСТИНГ ПРОГРАМНОГО КОДУ 56

ДОДАТОК Б МОКАПИ 57

Міністерство освіти і науки України
Державний університет «Одеська політехніка»
Інститут комп’ютерних систем
Кафедра інформаційних систем

Рівень вищої освіти: перший (бакалаврський)
Спеціальність: 122 – Комп’ютерні науки
Освітня програма: Комп’ютерні науки

ЗАТВЕРДЖУЮ
Завідувач кафедри ІС
_____
(підпис)
д.т.н., проф. Арсірій О.О.
« » _____ 20__ р.

ЗАВДАННЯ
НА КВАЛІФІКАЦІЙНУ РОБОТУ

Нікітіна Павла Яковича, група ЛАІ-195

1. Тема роботи: Розробка мобільного застосунку для збору та агрегації даних про вакансії

Керівник роботи: Петросюк Денис Валерійович, асистент

затверджені наказом ректора від 29.05.2023 р. № 317-в.

2. Зміст роботи: Вступ. Огляд існуючих систем збору та агрегації даних про вакансії. Проектування застосунку збору та агрегації даних про вакансії. Реалізація застосунку збору та агрегації даних про вакансії. Висновки. Перелік використаних джерел. Додатки.

3. Перелік ілюстративного матеріалу: Комп’ютерна презентація.

4. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

5. Дата видачі завдання 13.02.2023 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів кваліфікаційної роботи	Строк виконання етапів роботи	Примітка
1	Огляд систем-аналогів та технологій їх розробки	15.03.23	виконано
2	Проектування компонентів модулю управління системою опаленням багатоквартирного будинку	30.04.23	виконано
3	Програмна реалізація модулю управління системою опалення багатоквартирного будинку	15.05.23	виконано
4	Тестування програмного забезпечення. Оцінка працездатності модулю управління	25.05.23	виконано
5	Оформлення пояснювальної записки	01.06.23	виконано
6	Нормоконтроль		
7	Попередній захист		
8	Захист		

Здобувач вищої освіти _____ (підпис) Нікітін П.Я. (прізвище та ініціали)

Керівник роботи _____ (підпис) Петросюк Д.В. (прізвище та ініціали)

АНОТАЦІЯ

Нікітін П. Я. Розробка мобільного застосунку для збору та агрегації даних про вакансії (кваліфікаційна робота бакалавра). – Одесса: ДУ «Одеська політехніка», 2023. – *X* с.

Кваліфікаційну роботу бакалавра присвячено розробці мобільного застосунку, який допоможе користувачам у пошуках роботи. В роботі розкривається мета та актуальність даної розробки у сучасному часі та для конкретної цільової аудиторії. В кваліфікаційній роботі було здійснено порівняльний аналіз з існуючими аналогами та обрано технології розробки: мови Java та Kotlin з клієнт-серверною архітектурою та використанням Android SDK, REST API, SQLite. Також спроектовано та реалізовано даний мобільний застосунок.

ANNOTATION

Nikitin P. Development of a mobile application for collecting and aggregating data on vacancies (qualifying work of the bachelor). - Odesa: Odesa Polytechnic State University, 2023. - *X* p.

The bachelor’s qualification work is dedicate to the development of a mobile application that will help users in their job search. The paper reveals the purpose and relevance of this development in modern times and for a specific target audience. In the qualification work, a comparative analysis was performed with existing analogues and development technologies were selected: Java and Kotlin languages with client-server architecture and using the Android SDK, REST API, and SQLite. This information support was also designed and implemented.

ВСТУП

Працевлаштування стає дедалі актуальнішою проблемою. З кожним роком кількість безробітних зростає, а конкуренція на ринку праці лише посилюється. Щоб знайти роботу, необхідно витратити багато часу на пошук вакансій. Наприклад, в Україні існує декілька популярних сервісів, які надають вакансії. Коли кандидат шукає підходящу працю, йому потрібно відвідати усі ці сайти та додатки, на кожному з яких повторно заповнювати ти ж самі параметри, що збільшує час пошуку.

Метою даної дипломної роботи є розробка мобільного додатку, який допоможе користувачам швидко та зручно знаходити вакансії на ринку праці. Він буде надавати можливість пошуку вакансій за критеріями та навичками по всім найпопулярнішим сервісам, переглядати їх опис та зберігати вподобані вакансії.

Розробка програми передбачає використання сучасних технологій та інструментів, таких як Kotlin, Android SDK, REST API, SQLite та ін. В результаті роботи буде створено повноцінний продукт, який сприятиме покращенню пошуку роботи для користувачів.

Надалі робота буде включати аналіз існуючих рішень, проектування і розробку програми, а також його тестування та оцінку ефективності.

1 ОГЛЯД ІСНУЮЧИХ СИСТЕМ ЗБОРУ ТА АГРЕГАЦІЇ ДАНИХ ПРО ВАКАНСІЇ

1.1 Порівняння аналогів систем збору та агрегації даних про вакансії.

Для того, щоб зробити сучасний, надійний, функціональний та конкурентоспроможний продукт, необхідно зробити порівняння аналогів та визначити їх функціонал, переваги та недоліки. У даному випадку буде здійснюватися порівняльний аналіз сервісів, які надають інформацію щодо вакансій. Треба зазначити, що розроблюваний застосунок спрямований на IT-фахівців, тому будуть розглянути чотири сервіси, які найчастіше використовуються для пошуку праці у IT-галузі в Україні.

1.1.1 Jobs.ua

Jobs.ua – сайт для пошуку вакансій та послуг. На ньому можна обрати одну з 5 категорій:

- «Шукачу» – пошук вакансій за ключовими словами;
- «Роботодавцю» – сторінка сайту, на якої можна додати вакансію;
- «Освіта» – категорія, у якій розташовані курси з іноземної мови, коледжі та ВНЗ України, де можна переглянути їх контактні дані та напрямки навчання;
- «Тренінги» – список наявних тренінгів по містах України;
- «Публікації» – корисні статі, посилання законодавство України та інформація щодо створення резюме.

Аналіз буде відбуватися тільки за категорією «Шукачу», бо розроблюваний застосунок орієнтований саме на шукачів праці.

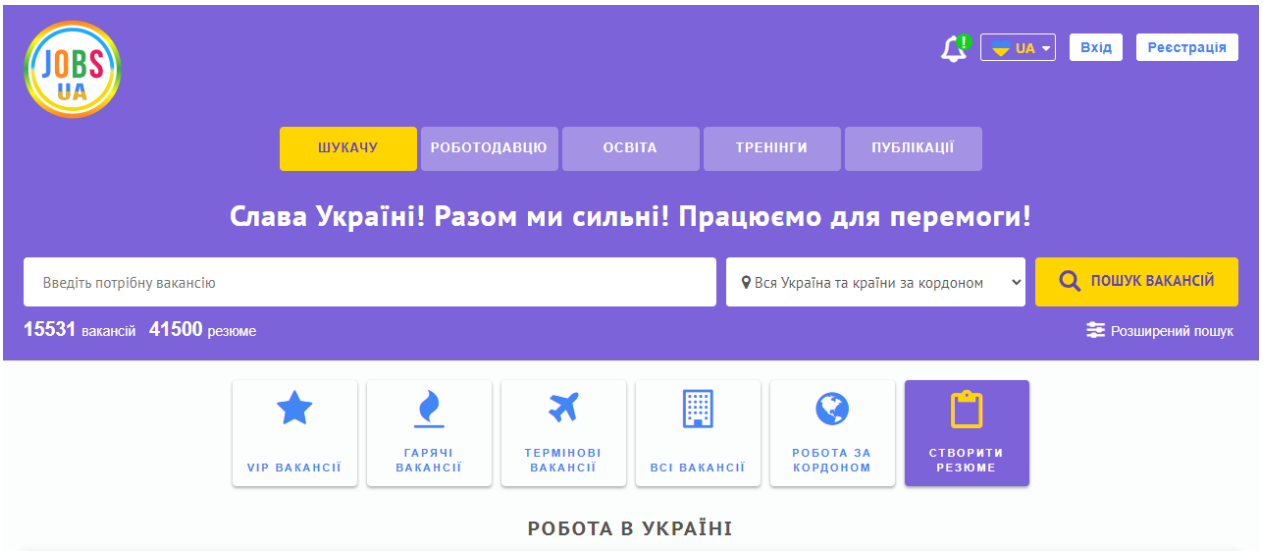


Рисунок 1.1 – Початкова сторінка сайту Jobs.ua

На сайті присутня можливість пошуку за ключовими словами та містом, а також розширений пошук, де можна шукати вакансії за досвідом роботи, категорією та зарплатою.

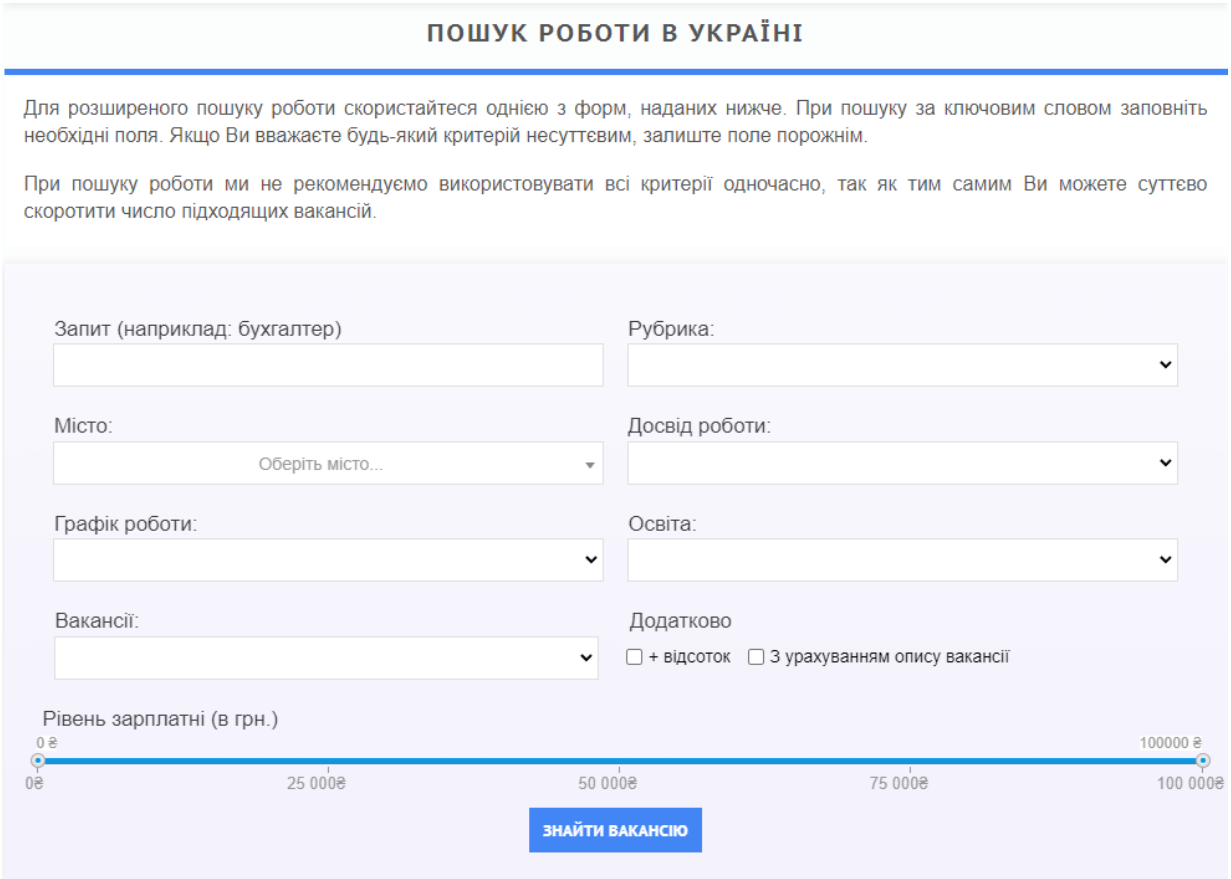


Рисунок 1.2 – Форма розширеного пошуку сайту Jobs.ua

При здійснені пошуку та наявних вакансій за критеріями, відображається список вакансій, де є назва вакансії, компанія, яка надає вакансію, місто, короткий опис та зарплатня, якщо наявна.

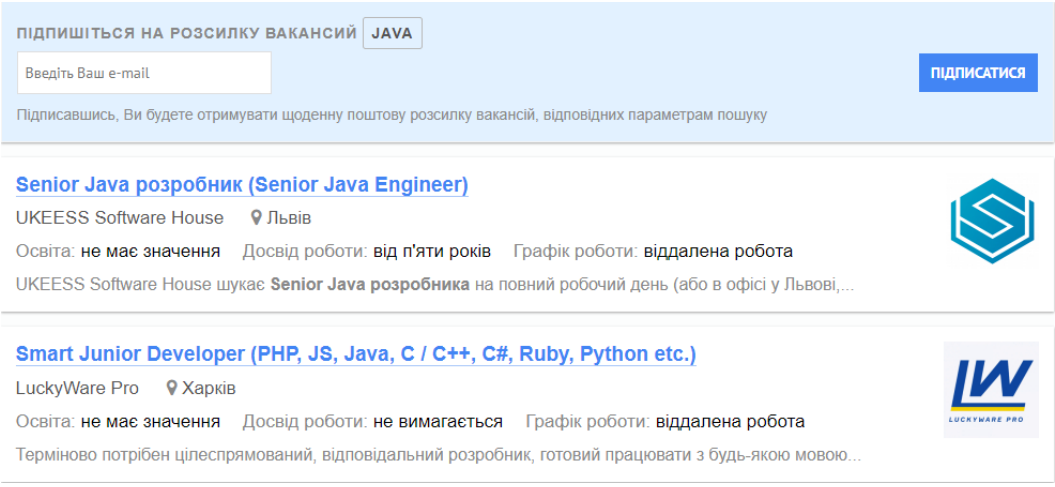


Рисунок 1.3 – Список вакансій сайту Jobs.ua

При натисканні на назву вакансії, відкривається сторінка вакансії де наявна інформація з списку вакансій та її детальний опис вакансії.

Рисунок 1.4 – Сторінка вакансії сайту Jobs.ua

До особливостей сайту можна віднести можливість завантаження опису вакансії у PDF-формату.

1.1.2 Work.ua

Work.ua – сайт для пошуку праці. На сайті присутня можливість пошуку за посадою або компанією та містом. Є можливість переглянути вакансії конкретної компанії, натиснувши на відповідний логотип. Також можна зареєструватися та розмістити своє резюме, за яким буде відбуватися розсилка вакансій.

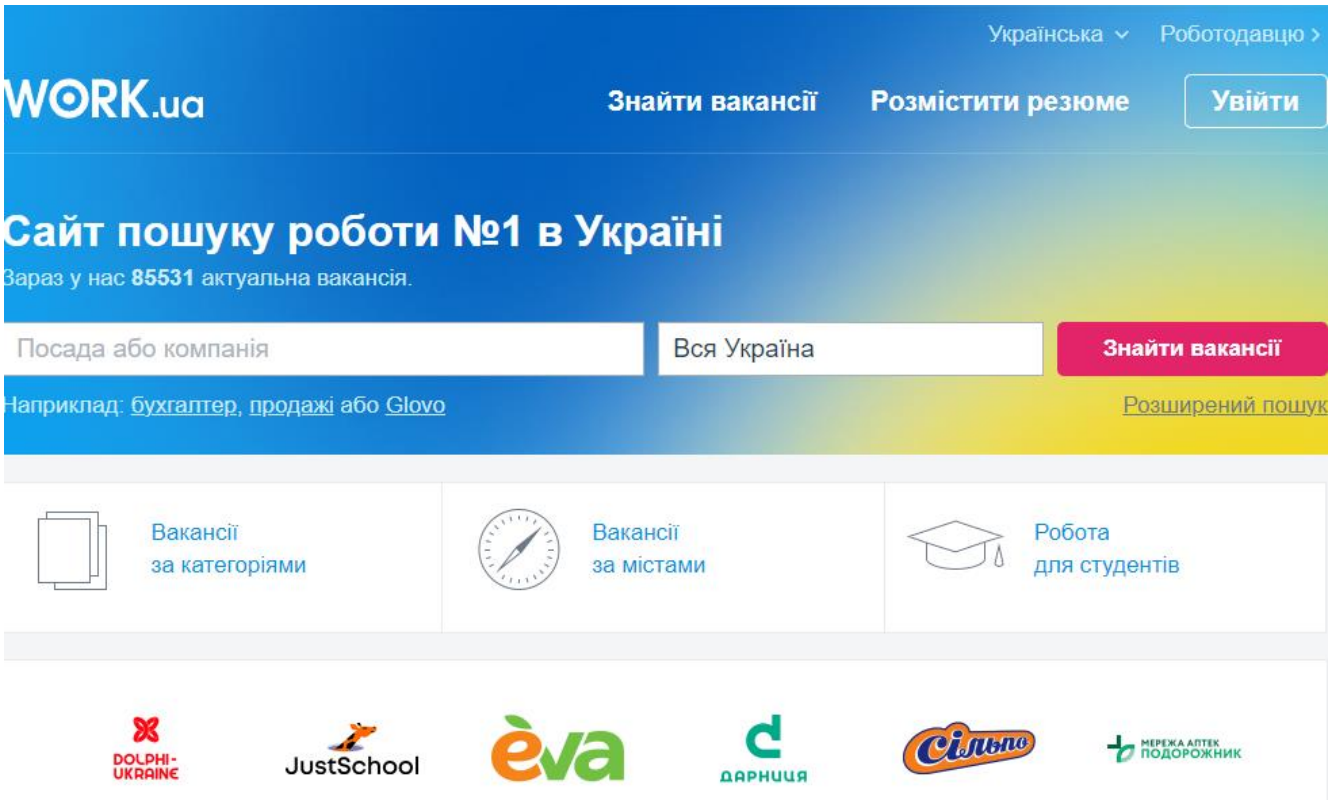


Рисунок 1.5 – Початкова сторінка сайту Work.ua

Є розширений пошук, де можна шукати вакансії за категоріями, зарплатою та соціальною групою, наприклад:

- Кандидатам без досвіду;
- Кандидатам без резюме;
- Студентам;
- Людям з інвалідністю;

– Ветеранам;

– Пенсіонерам.

Також, є можливість переглянути середню зарплатню за категорією.

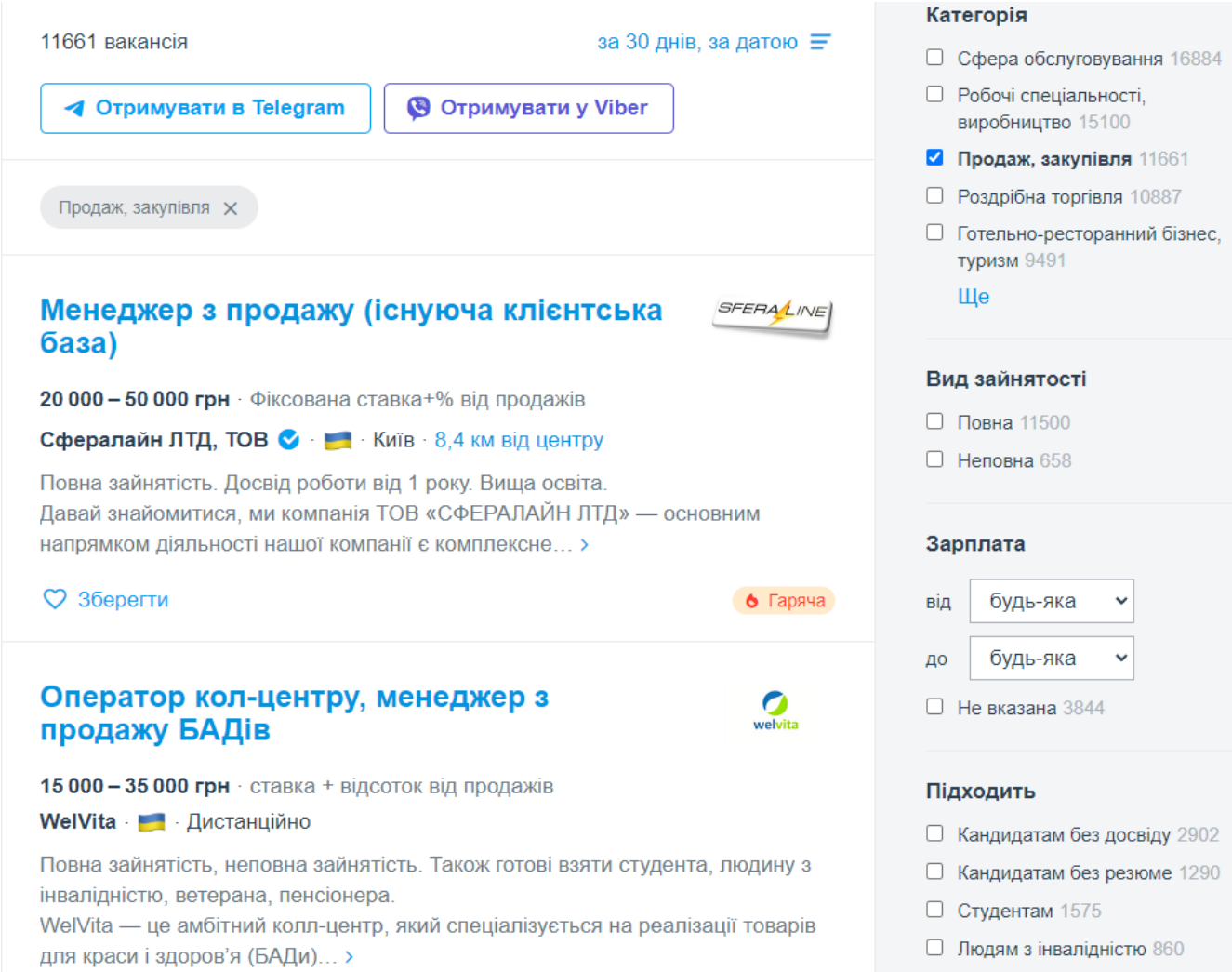
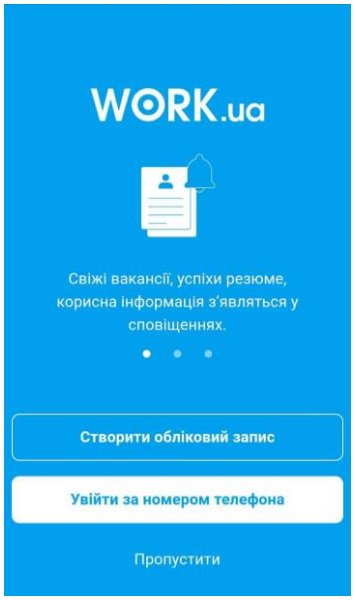


Рисунок 1.6 – Розширений пошук сайту Work.ua

Сайт Work.ua має мобільний застосунок, тому далі буде огляд саме нього. При першому запуску застосунку, буде запропоновано створити обліковий запис, увійти за номером телефону або пропустити вхід до облікового запису (рис.1.7 (а)). Розглянемо функціонал застосунку без входу до облікового запису. На екрані пошуку вакансій є поле для введення посади, можна обрати бажане місто та соціальну групу (рис.1.7 (б)). Внизу є панель навігації до наступних екранів:

– Пошук;

– Рекомендовані вакансії;
– Збережені вакансії;
– Сповіщення;
– Обліковий запис.

а) 

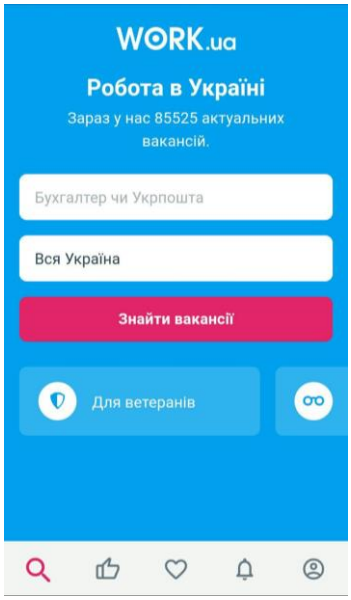
б) 

Рисунок 1.7 – а) початковий екран; б) екран пошуку вакансій

Щоб скористатися екранами «Рекомендовані вакансії», «Збережені вакансії» та «Сповіщення», необхідно увійти до облікового запису (рис.1.8).

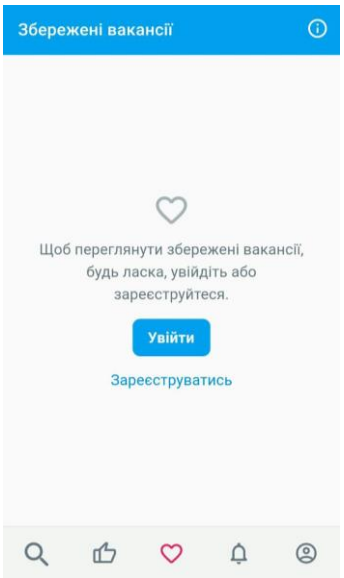
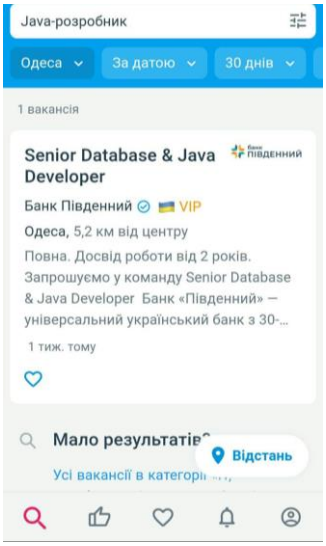
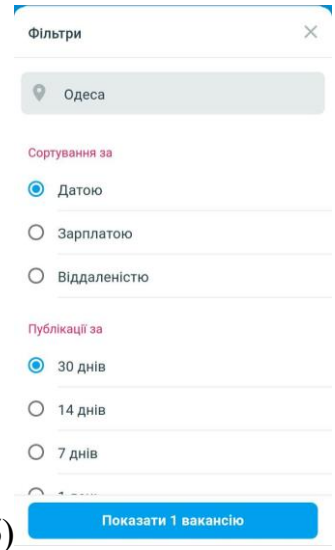


Рисунок 1.8 – екран «Збережені вакансії»

Розпочнемо пошук вакансій. На екрані пошуку почнемо вводити назву посади. У застосунки є автодоповнення, що можна віднести до переваг. Після натискання кнопки «Знайти вакансію» (рис.1.7 (б)), відкривається екран з введеною посадою та списком вакансій (рис.1.9 (а)). Якщо натиснути на кнопку параметрів у полі введення посади, відкривається налаштування фільтрів, де можна обрати спосіб фільтрації, категорії вакансій та зарплатню (рис.1.9 (б)).



а)



б)

Рисунок 1.9 – а) список вакансій; б) налаштування фільтрів

При натисканні на вакансію, відкривається докладний опис цієї вакансії (рис.1.10).

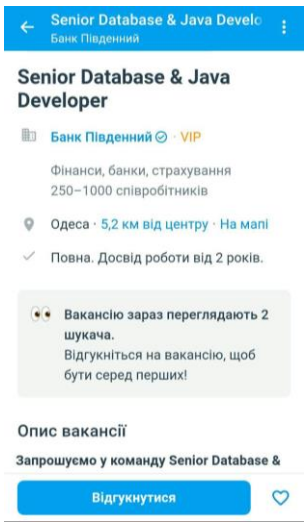


Рисунок 1.10 – Докладний опис вакансії

1.1.3 Djinni

Djinni – сайт для пошуку вакансій, але особливий. Основна ідея проекту – шукають вас, а не ви. Ця ідея складається в тому, на відміну від інших сервісів пошуку вакансій, що профіль рекрутерів відкритий, а профіль користувача – закритий. Але з цього виникає недолік – користуватися проектом можна тільки зареєстрованим користувачам.

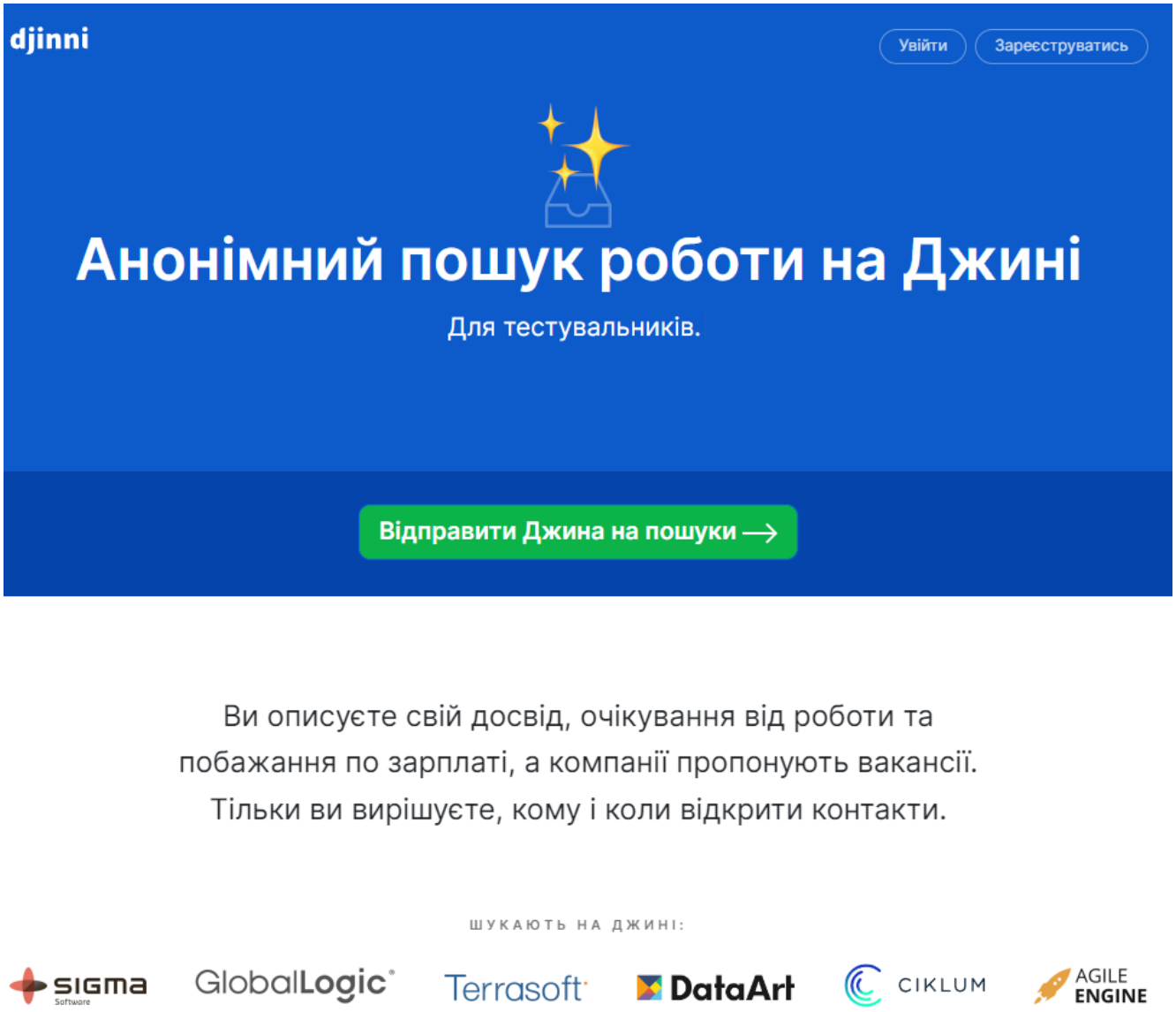


Рисунок 1.11 – Початкова сторінка сайту Djinni

Зареєструватися на сайті можна через відповідну форму реєстрації, або можна увійти за допомогою аканту Google або LinkedIn.

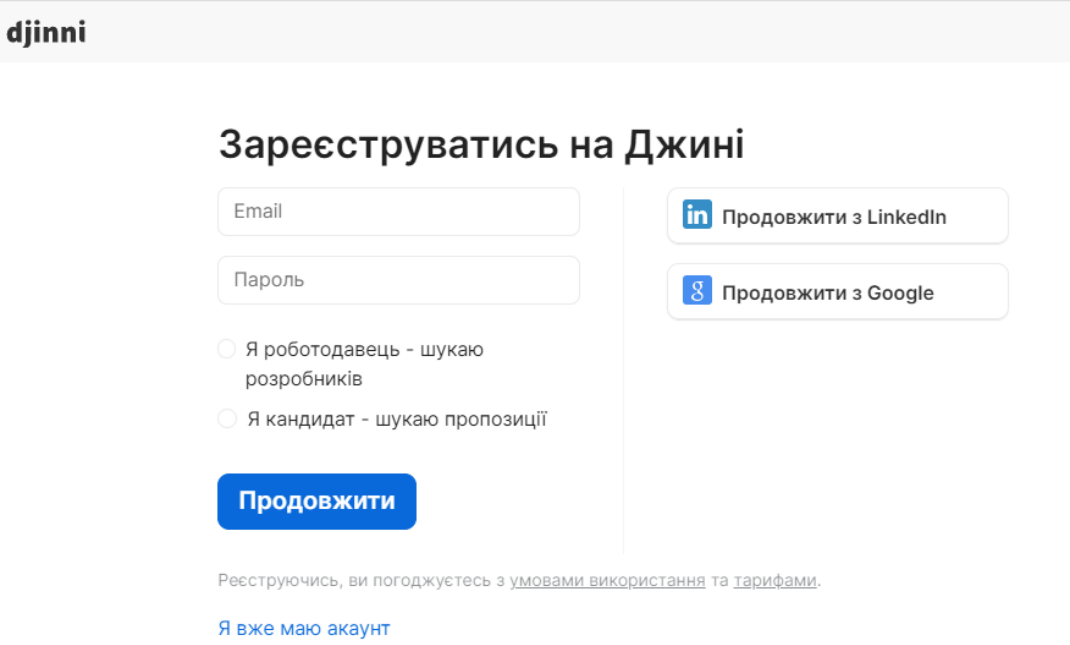


Рисунок 1.11 – Реєстрація на сайті Djinni

Для того, щоб розпочати пошук необхідно заповнити профіль користувача, у якому вказати свій досвід, очікувану зарплатню та спеціалізацію.

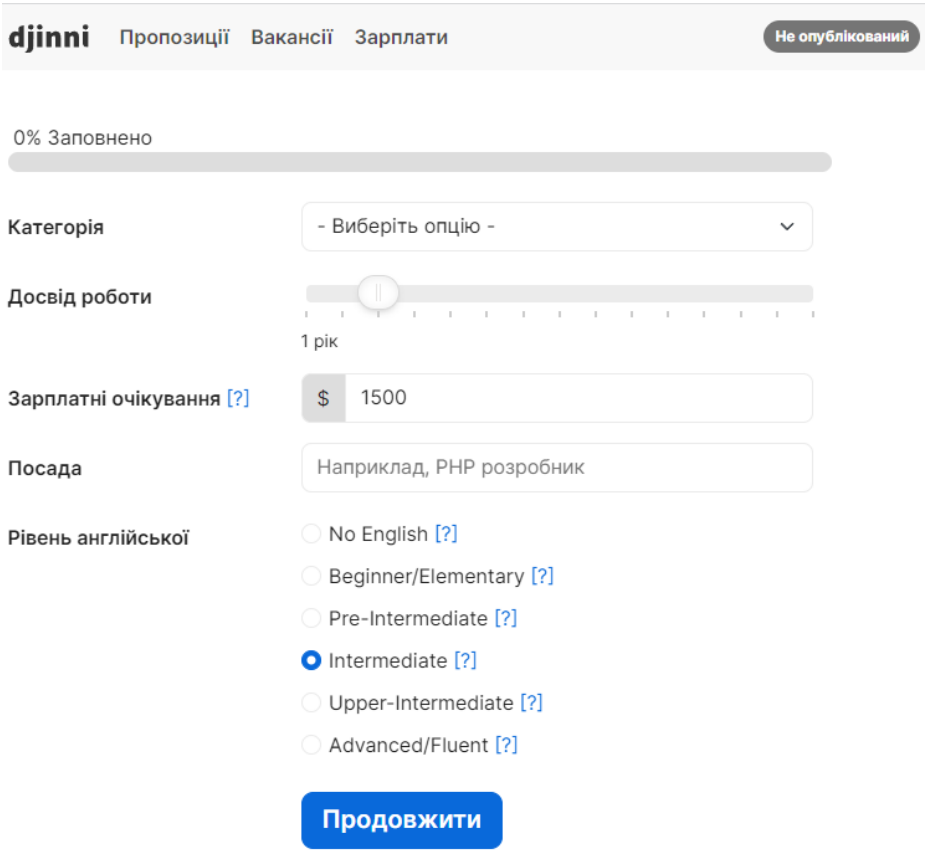


Рисунок 1.12 – Заповнення профілю на сайті Djinni

Після заповнення профілю, будуть доступні опції з пошуку вакансій:

– «Для вас» - список вакансій, які відповідають профілю;
– Всі – список усіх вакансій сервісу.

Шукати вакансії можна за ключовими словами. Присутня можливість встановлення параметрів пошуку, таких як:

– Спеціалізація;
– Країна;
– Зайнятість;
– Зарплатня;
– Тип компанії.

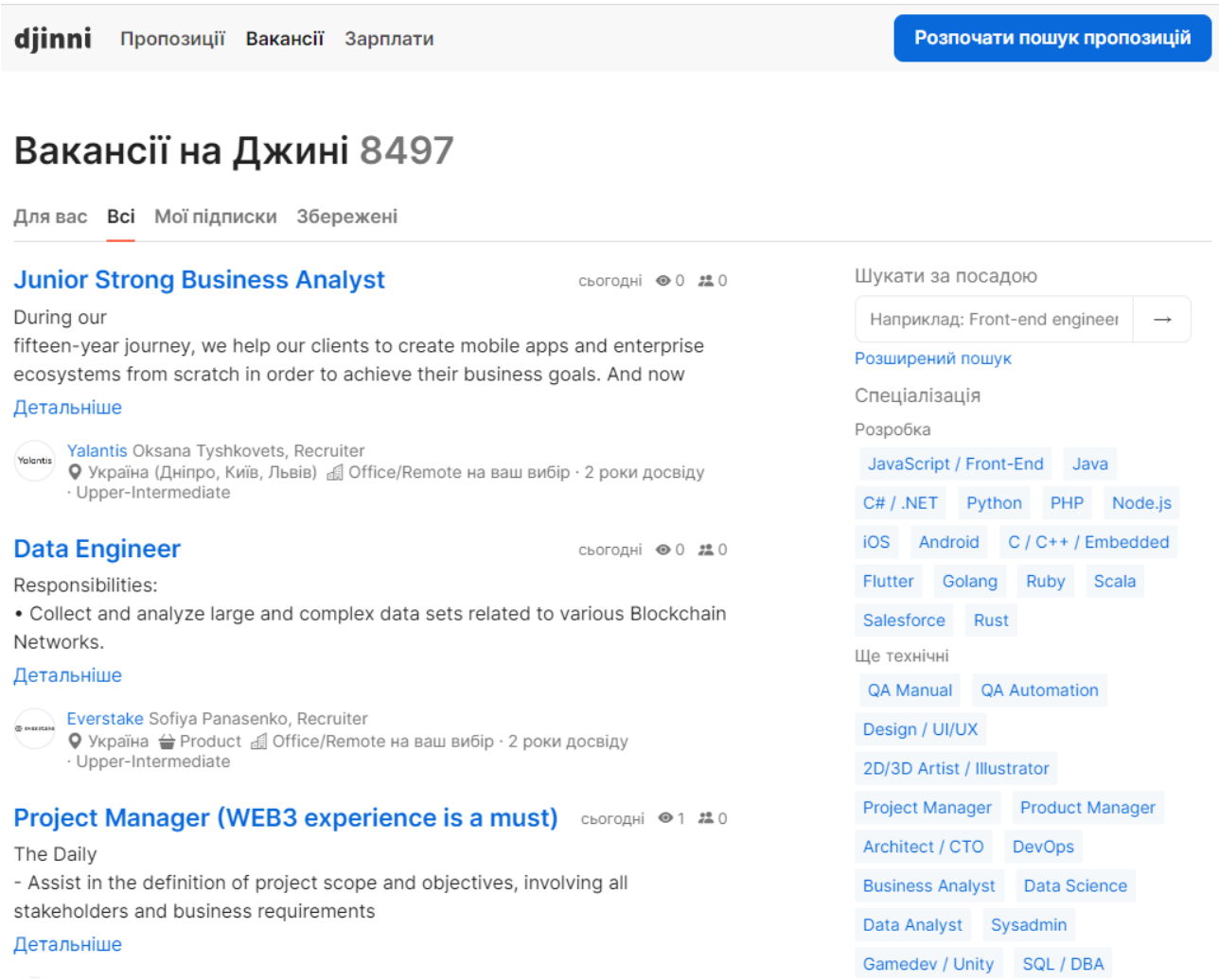


Рисунок 1.13 – Список вакансій на сайті Djinni

При натисканні на вакансію відкривається її докладний опис.

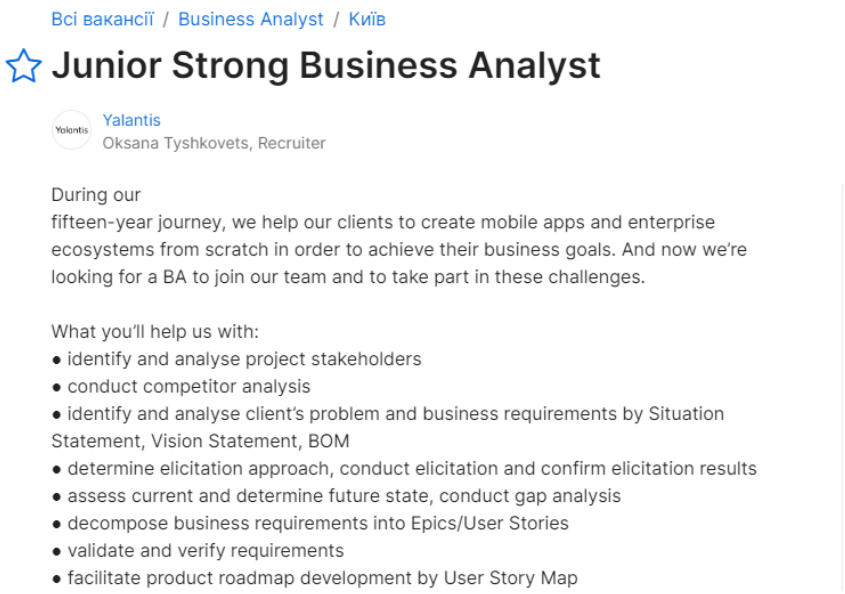


Рисунок 1.14 – Докладний опис вакансії на сайті Djinni

До особливостей сайту можна віднести наявність інформації щодо зарплатні по різним категоріям. Можна обрати категорію та часовий проміжок і отримати звіт щодо зарплатні.

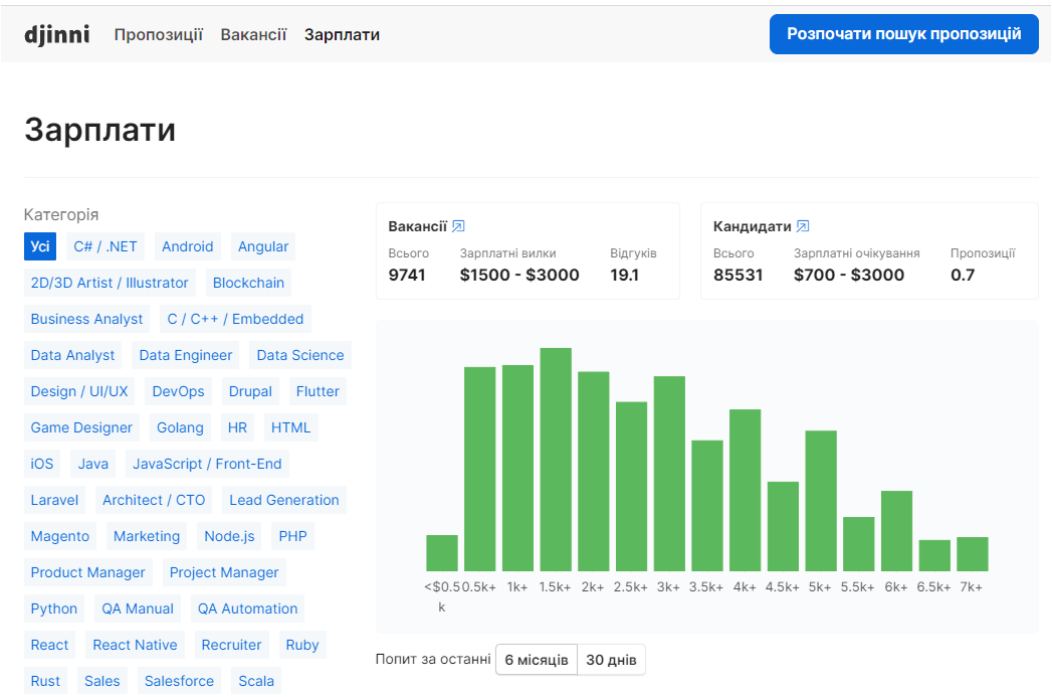


Рисунок 1.15 – Звіт про зарплатню на сайті Djinni

1.1.4 Dou.ua

Dou.ua – це величезний портал, пов’язаний з IT в Україні. На ньому присутні новини галузі, форум користувачів, звіт про зарплати в галузі та є можливість пошуку вакансії, яка і буде розглядатися.

Рисунок 1.16 – Головна сторінка сайту Dou.ua

У розділі «Робота» є можливість пошуку вакансій. Є пошук за категоріями, ключовими словами та перегляд вакансій від конкретних компаній.

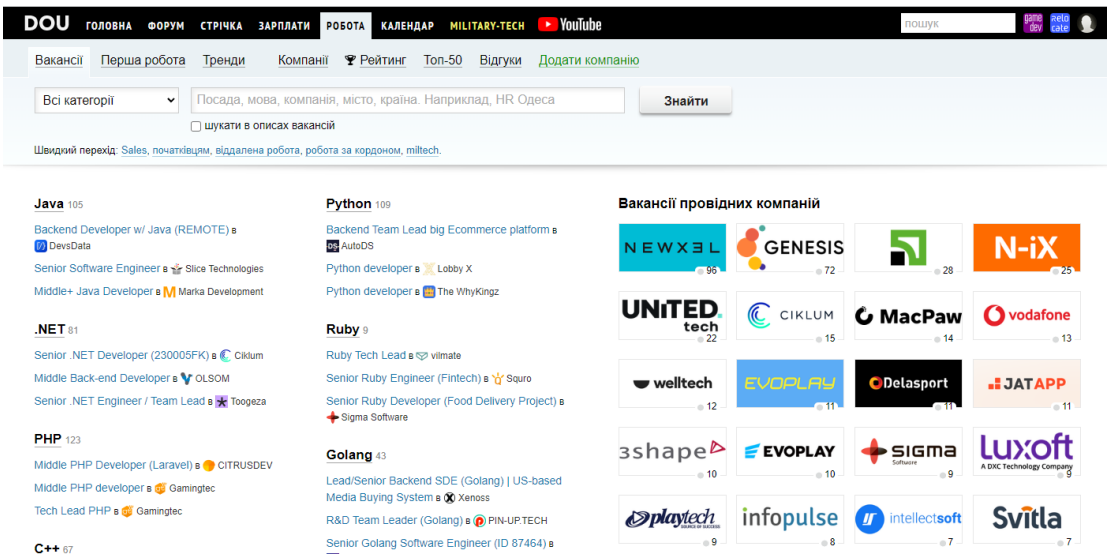


Рисунок 1.16 – Розділ «Робота» сайту Dou.ua

Після введення ключових слів та/або обирання категорії, буде надано список вакансій. Можна встановити фільтри за досвідом та містом.

Рисунок 1.17 – Список вакансій сайту Dou.ua

При натисканні на вакансію, відривається її докладний опис.

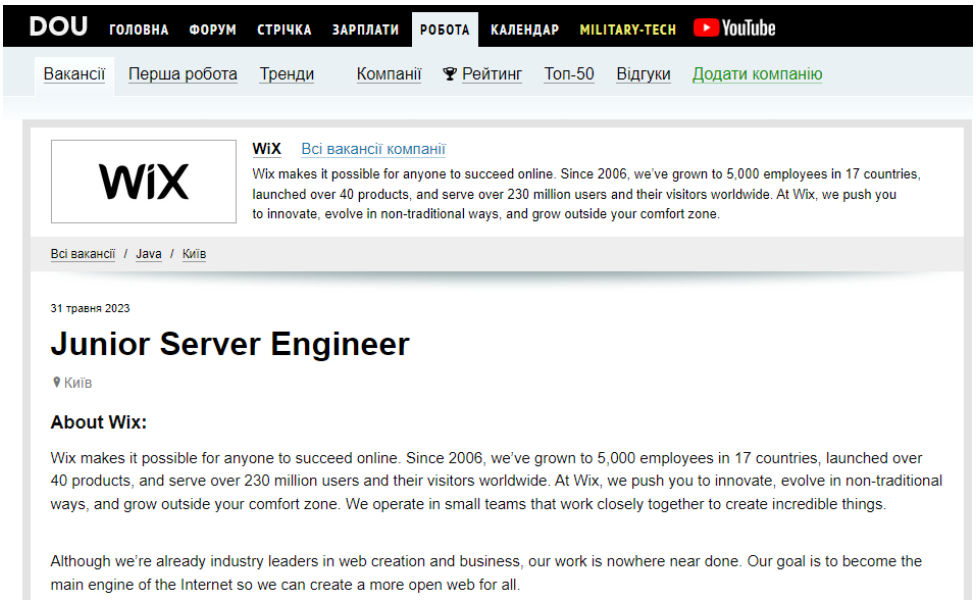


Рисунок 1.18 – Докладний опис вакансії сайту Dou.ua

1.1.5 Результат порівняння

Для результату аналізу функціоналу сервісів, створимо порівняльну таблицю (табл.1.1), в ній представимо найбільш важливі критерії функціоналу для порівняння аналогів сервісів для пошуку вакансій.

Таблиця 1.1 – Порівняльна таблиця аналогів

Критерії	Сервіси				
	Jobs.ua	Work.ua	Djinni	Dou.ua	Розробляємий застосунок
Пошук за ключовими словами	+	+	+	+	+
Пошук за локацією	+	+	+	+	+
Пошук за категорією	+	+	+	+	+
Можливість збереження вакансії	+	+	+	-	+
Можливість пошуку без профілю	+	+	-	+	+
Обмеження функціоналу без профілю	-	+	+	-	-
Наявність мобільного застосунку	-	+	-	-	+

Підбиваючи підсумки, можна сказати, що розроблюваний застосунок є конкурентоспроможним. Наявні усі необхідні параметри пошуку, немає обмеження функціоналу без створеного профілю та займається незаповнена ніша мобільних застосунків, бо тільки один з ведучих сайтів для пошуку вакансій має мобільний застосунок. Також, треба розуміти, що для успіху продукту, необхідно щоб продукт мав унікальний функціонал, який би приваблював користувача та якісно відрізняв продукт від аналогів. У цьому випадку, унікальний функціонал – можливість пошуку вакансій за навичками користувача. Шукач вводить свої навички, система аналізує їх та повертає список вакансій, до яких ці навички відносяться.

1.2 Вибір технологій для розробки системи

Система складається з трьох основних частин:

- Android застосунок (клієнт);
- Сервер;
- Модель для аналізу даних.

Для кожної частини є свій стек технологій, який найліпше підходить до вирішення відповідного завдання. Наприклад, усі частини можна створити на підставі Java та її бібліотек і фрейморків, але для моделей, які аналізують дані, це не буде зручним та ефективним підходом, тому що існують технології, завдяки яким ця задача вирішується набагато легше. Тому, для кожної частини системи необхідно обрати свій відповідний стек технологій, який надасть змогу вирішити задачу якомога швидше, легше та ефективніше.

1.2.1 Вибір мови програмування

Клієнтська частина.

Для створення цієї частини було обрано мову програмування Kotlin. Kotlin – це мова програмування, яка розроблена для роботи на платформі Java Virtual Machine (JVM). Він став популярним серед розробників Android, тому що він

пропонує синтаксис, який виразніший і зрозуміліший, ніж Java. Ось деякі з переваг використання Kotlin для розробки програм під Android:

- підтримка сумісності з Java – Kotlin є повністю сумісним з Java і може використовуватися разом з Java-кодом, що вже існує у програмі. Це означає, що ви можете поступово переходити на Kotlin, не переписуючи весь свій код;
- більш безпечний код – Kotlin включає безпечну перевірку типів і нульових значень, що дозволяє уникнути помилок, пов'язаних з нульовими посиланнями. Це також означає, що ваш код буде більш читабельним та зрозумілим;
- код із меншою кількістю рядків – Kotlin дозволяє скоротити кількість коду, необхідного для виконання тієї ж роботи, що і Java. Це означає, що ви можете писати код швидше та ефективніше;
- Kotlin має бібліотеку Kotlinx Coroutines, яка дозволяє писати простий, зрозумілий і потоково-безпечний асинхронний код.

Зараз Kotlin став цільовою мовою програмування для Android, що пропонує компанія Google.

Серверна частина.

Для серверної частини була обрана мова програмування Java. Java є строго типізованою, об’єктно-орієнтованою мовою програмування, що дозволяє створювати чітко структурований, типобезбечний, зрозумілий код. Java є однією з найпопулярніших мов програмування, завдяки чому існує багато допоміжних матеріалів та велика підтримка від суспільства – безліч бібліотек та фреймворків, які спрощують вирішення багатьох задач. Також, Java зарекомендувала себе, як одне з найкращих рішень для створення надійних серверів у великих компаніях.

Моделі для аналізу даних.

Майже стандартом для вирішення задач аналізу даних є мова програмування Python. Python – динамічно-типізована, об’єктно-орієнтована мова програмування. Він відрізняється своєю легкістю та гнучкістю, що дозволяє дуже швидко та просто

створювати програмний код. Також, Python значно простіший до освоєння, у порівнянні з іншими мовами програмування, що привабило багато професіоналів з інших галузей, та призвело до появи багатьох інструментів для вирішення наукових та математичних задач. Саме ця різноманітність інструментів та велика підтримка від суспільства призвела до того, що у Python найбільш привабливі можливості для вирішення задач аналізу даних.

1.2.2 Вибір середовища програмування

Клієнтська частина.

Для розробки клієнтської частини було обрано інтегроване середовище розробки Android Studio. Це середовище розроблено компанією Google у співпраці з компанією JetBrains, і призначене для розробки Android-додатків. Вона має величезний функціонал: автоматичне форматування коду, вказівку користувачеві на помилки, можливість виправлення помилок засобами IDE, інтеграція з системами контролю версій, створення структури проекту, створення шаблонних проектів, робота з ресурсами, наявність візуального відображення UI програми, можливість роботи та тестування програми на реальному пристрою або емуляторі, створення та налагодження проекту, можливість отримувати інформацію з пристрою про завантаженість процесора, споживання оперативної та постійної пам'яті, можливість моніторингу процесів, запущених на пристрої тощо.

Серверна частина.

Для розробки серверної частини було обрано середовище розробки Intellij IDEA, створене компанією JetBrains для розробки на мовах програмування Java, Kotlin та інших. Це середовище розробки має дві версії:

- Community edition – безплатна версія, з меншим функціоналом;
- Ultimate edition – платна версія, з підтримкою створення Java Spring проектів, Web-проектів, які використовують мову відмінну від Java та Kotlin, більшу кількість фреймворків та призначена для розробки комерційних продуктів[1].

Для створення системи була обрана Community edition, бо вона надає увесь необхідний функціонал. Середовище розробки Intellij IDEA має ті самі переваги, що й Android Studio, бо це приблизно одне й теж саме середовище, але для різних вимог.

Модель для аналізу даних.

Для створення моделі аналізу даних було використано середовище розробки Jupyter Notebook. Воно призначене для створення коду на мові програмування Python, де відразу можна бачити результат виконання коду та його окремих фрагментів. На відміну від традиційного середовища розробки, код можна розбити на шматки та виконувати їх у довільному порядку, що дуже корисно при тестуванні коду та експериментів з ним, що є одним з найголовніших етапів при створенні моделі для аналізу даних.

1.2.3 Вибір фреймворків та бібліотек

Клієнтська частина.

Для розробки використовувався фреймворк Android Jetpack, що розширює стандартні засоби та функціонал Android SDK, що надає зручні та ефективні засоби для роботи з навігацією, зберіганням даних, фоновими процесами та багато іншого. Також, для тестування був використаний фрейморк JUnit.

Набір бібліотек, які були використані:

- Material Components – бібліотека, що надає сучасні та безпечні компоненти для створення інтерфейсу користувача. Google рекомендує використання цих компонентів, оскільки вони безпосередньо інтегровані з можливостями ОС і надають можливість розширення та зміни візуальних компонентів, що вкрай зручно, адже вони оптимізовані та протестовані, що полегшує життя розробнику;
- Retrofit – бібліотека для спілкування з RESTful веб-сервісами, яка бере на себе відповідальність за створення запитів та обробку відповідей[2];

- Moshi - сучасна бібліотека для обробки JSON для Android, Java і Kotlin. Вона потрібна для перетворення об’єктів JSON на Java або Kotlin класи[3];
- Room – бібліотека, яка забезпечує рівень абстракції над SQLite, щоб забезпечити вільний доступ до бази даних, одночасно використовуючи всю потужність SQLite;
- Kotlinx Coroutines – бібліотека, що дозволяє працювати з корутинами в Android, використовуючи мову Kotlin. Корутини дуже потужний інструмент для вирішення асинхронних завдань, в даному випадку, звернень до бази даних та виконання запитів до сервера;
- Mockk – це бібліотека для створення та управління макетами об'єктів (mock objects) у тестах для програм Android. Вона надає можливість створювати заглушки (mocks) для об'єктів та контролювати їхню поведінку, щоб перевіряти коректність роботи програми у різних сценаріях. За допомогою Mockk можна створювати заглушки для будь-яких об'єктів, включаючи класи, інтерфейси, сінглтони та інші.

Серверна частина

Для серверної частини був обран фреймворк Spring. Spring Framework — це структура програми та інверсія контейнера керування для платформи Java. Основні функції фреймворку можуть використовуватися будь-якою програмою Java, але існують розширення для створення веб-програм на основі платформи Java EE (Enterprise Edition). Хоча фреймворк не нав’язує жодної конкретної моделі програмування, він став популярним у спільноті Java як доповнення до моделі Enterprise JavaBeans (EJB) [6].

Бібліотеки, які використовувалися:

- Jsoup - бібліотека, що надає функціонал для парсингу сайтів. У ній вже є вбудовані механізми отримання HTML-сторінок, а також засоби для роботи з HTML, такі як пошук елементів за назвою, пошук елементів за класами або ідентифікаторами, можливість отримання значень атрибутів та вмісту елементів та багато іншого;

– Gson - це бібліотека Java, яку можна використовувати для перетворення об’єктів Java у їх представлення JSON[5].

Модель для аналізу даних.

Для створення моделі для аналізу даних були обрані наступні бібліотеки:

– Pandas - це бібліотека, яка забезпечує швидкі, гнучкі та виразні структури даних, розроблені для того, щоб зробити роботу з «реляційними» або «міченими» даними одночасно легкою та інтуїтивно зрозумілою[4];
– scikit-learn - це бібліотека машинного навчання. Вона надає інструменти для різних завдань машинного навчання, включаючи класифікацію, регресію, кластеризацію, зниження розмірності, вибір моделей та попередню обробку даних;
– imblearn - це бібліотека для обробки та вирішення проблеми незбалансованих даних у машинному навчанні. Вона надає різні методи роботи з незбалансованими наборами даних, включаючи методи семплювання, зниження розмірності та класифікації.

1.2.4 Вибір системи управління базами даних

Клієнтська частина.

Як систему управління базою даних було обрано SQLite. SQLite є локальною вбудовуваною СУБД, яка може бути використана як база даних для додатків під Android. SQLite доступна за промовчанням в Android SDK і надає ефективне зберігання та вилучення даних. Докладніше про цю СУБД:

– SQLite використовує мову SQL для роботи з базою даних. Ця мова запитів підтримує безліч операцій, таких як вибірка даних, додавання, оновлення та видалення даних із таблиць;
– SQLite є СУБД, що вбудовується, що означає, що вона не працює як окремий сервер, а інтегрується в додаток. Таким чином, SQLite не потребує встановлення та налаштування, що спрощує її використання;
– SQLite забезпечує швидке зберігання та вилучення даних завдяки своїй внутрішній архітектурі, яка оптимізована для роботи з локальними базами

даних. Крім того, SQLite забезпечує підтримку індексів, що прискорює вибірку даних;

– SQLite може використовуватися для зберігання невеликих та середніх обсягів даних. Для великих обсягів даних може знадобитися використання іншої СУБД;
– SQLite забезпечує безпеку даних завдяки використанню файлового формату, що підтримує безліч функцій захисту даних, таких як шифрування;

Загалом SQLite - це ефективна і проста у використанні локальна СУБД, яка може бути використана в додатках під Android для зберігання та вилучення даних.

Серверна частина.

Для серверної частини була обрана СУБД PostgreSQL. PostgreSQL є потужною та широко використовуваною системою управління базами даних з відкритим вихідним кодом. PostgreSQL відомий своєю надійністю та стабільністю. Він має механізми перевірки цілісності даних, підтримку транзакцій та багатоверсійність, що забезпечує безпечне та надійне зберігання даних. Також, існує багато інструментів та бібліотек, які полегшують роботу з PostgreSQL у Java-додатках на базі Spring. Наприклад, Spring Data JPA надає зручний спосіб взаємодії з базою даних, а Spring Boot автоматизує конфігурацію та розгортання програми.

1.2.5 Вибір патернів проектування

Клієнтська частина.

Model-View-ViewModel (MVVM) – це арзітектурний патерн програмного забезпечення, який долає всі недоліки патернів проектування MVP та MVC. MVVM пропонує відокремити логіку представлення даних (Views або UI) від основної бізнес-логіки додатку[7].

Окремими шарами коду MVVM є:

– Model – цей рівень відповідає за абстракцію джерел даних. Model і ViewModel працюють разом, щоб отримати і зберегти дані;

– View – інформує ViewModel про дії користувача. Цей рівень спостерігає за ViewModel і не містить жодної прикладної логіки;
– ViewModel – обробляє дані та дії користувача, отримані від View. Служить сполучною ланкою між моделлю і представленням.

Observer – це поведінковий патерн проектування, який дозволяє визначити механізм підписки для сповіщення декількох об'єктів про будь-які події, що відбуваються з об'єктом, за яким вони спостерігають[8]. У даній системі цей патерн використовується для сповіщення представлення о набутті ViewModel списку вакансій від серверу або з локальної бази даних.

Серверна частина.

Model View Controller (MVC) – шаблон проектування який визначає, що застосунок складається з моделі даних, інформації для представлення та інформації для керування. Шаблон вимагає, щоб кожна з цих частин була відокремлена в окремі об'єкти[9]. Розглянемо його составні:

– Model – містить лише чисті дані додатку, вона не містить логіки, що описує, як представити дані користувачеві;
– View – представляє дані моделі користувачеві. Представлення знає, як отримати доступ до даних моделі, але воно не знає, що ці дані означають або що користувач може зробити, щоб маніпулювати ними.
– Controller – знаходиться між представленням і моделлю. Це місце, де написана фактична бізнес-логіка. Він прослуховує події, викликані представленням (або іншим зовнішнім джерелом), і виконує відповідну реакцію на ці події. У більшості випадків реакція полягає у виклику методу на моделі.

Для свторення системи було обрано архітектурний шаблон «Клієнт-сервер».

1.3 Висновки до першого розділу

В першому розділі даної кваліфікаційної роботи було проведено аналіз функціоналу аналогів сервісів, що надають вакансії. В результаті можна сказати,

що розробляємий застосунок є конкурентноспроміжним, за рахунок надання можливості пошуку за різними параметрами, повним функціоналом без необхідності створення профилю та завдяки унікальному фунцкіоналу, а саме пошуку вакансій за навичками шукача.

Застосунок розроблений для пристроїв на базі операційної системи Android. Розробка клієнтської частини системи була здійснена на мові програмування Kotlin, з використанням архітектурного шаблону MVVM. Розробка здійснювалася у середовищі розробки Android Studio. Для зберігання інформації була використана система управління базами даних SQLite. Серверна частина розроблена на мові програмування Java з використанням архітектурного шаблону MVC, у середовищі розробки Intellij IDEA Community edition з використанням фреймворку Spring та СУБД PostgreSQL. Модель, що аналізує дані розроблена на мові програмування Python з використанням сучасних бібліотек для машинного навчання.

2 ПРОЕКТУВАННЯ ЗАСТОСУНКУ ЗБОРУ ТА АГРЕГАЦІЇ ДАНИХ ПРО ВАКАНСІЇ

2.1 Мета та задачі ІС

Мета ІС: розробка Android – застосунку для шукача праці, де він може знайти вакансії, переглянути та зберегти вакансії, які підходять йому за його параметрами пошуку та навичками.

Цільова аудиторія: шукачі праці.

Головні функції застосунку:

- отримання вакансій з сайтів, які їх розміщують;
- пошук вакансій за ключовими словами, параметрами та навичками;
- перегляд списку вакансій;
- перегляд докладного опису вакансії;
- збереження вподобаних вакансій.

Інформаційні потоки ІС:

Функціонал «Пошук вакансій за параметрами»:

Модуль отримує на вхід дані про ключові слова пошуку, категорію, досвід та локацію. Формується та надсилається запит до серверу, після чого формується список вакансій, який відображається користувачу.

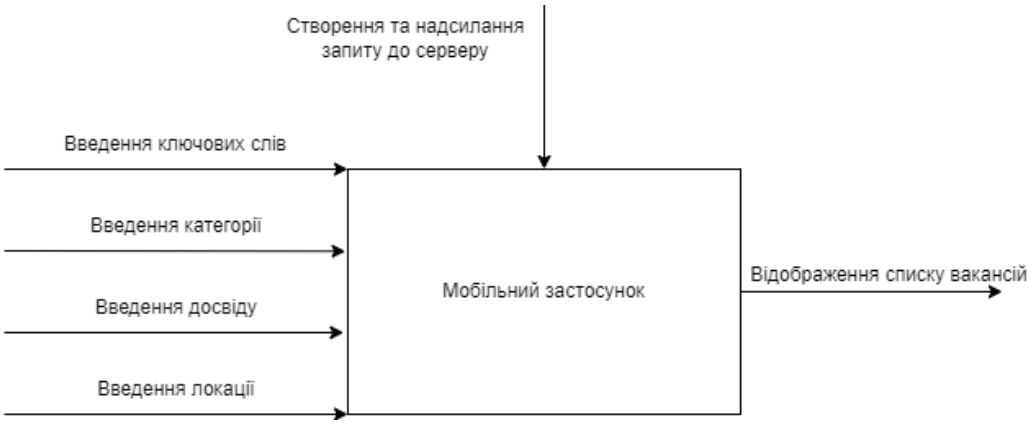


Рисунок 2.1 – Інформаційні потоки функціоналу «Пошук вакансій за параметрами» для Android застосунку

Модуль отримує запит від клієнта, після чого передає дані скраперам та запускає їх. Після виконання роботи скраперів, формується відповідь, яка надсилається клієнту.

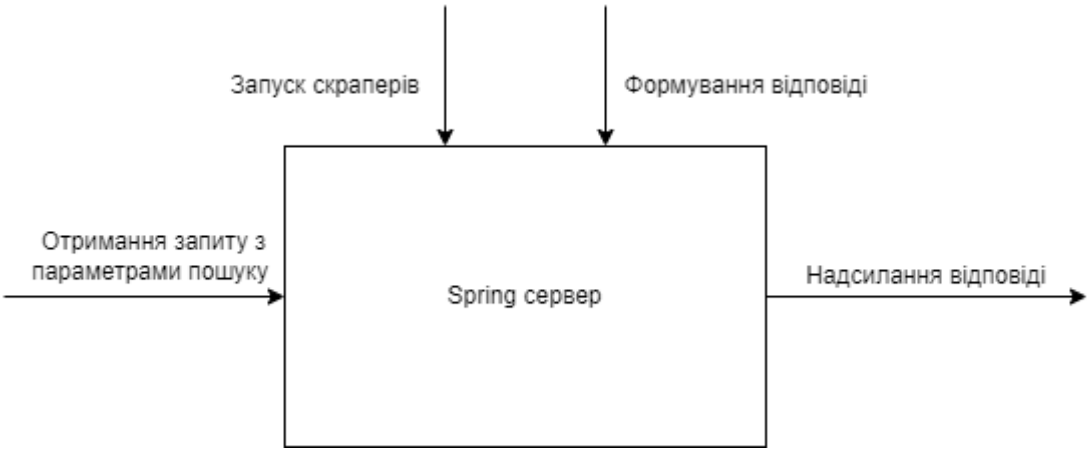


Рисунок 2.2 – Інформаційні потоки функціоналу «Пошук вакансій за параметрами» для Spring серверу

Функціонал «Пошук вакансій за навичками»:

Модуль отримує на вхід навички користувача. Формується та надсилається запит до серверу, після чого формується список вакансій, який відображається користувачу.

Рисунок 2.3 – Інформаційні потоки функціоналу «Пошук вакансій за навичками» для Android застосунку

Модуль отримує запит від клієнта, які передає в модель для аналізу даних. За отриманими даними від моделі, відбувається звернення до бази даних та отримання відповідних вакансій. Після отримання даних від бази даних, формується відповідь, яка надсилається клієнту.

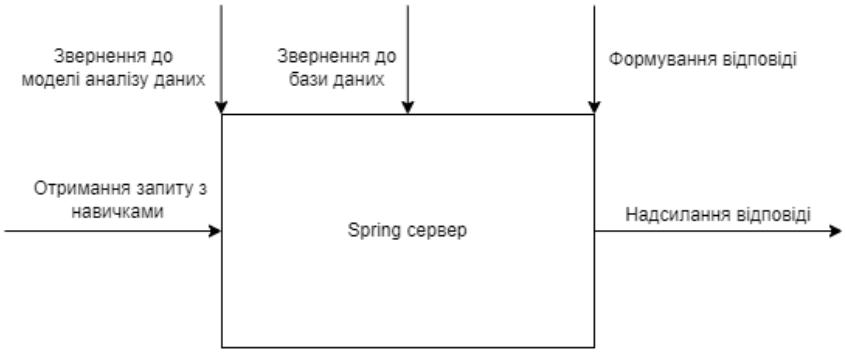


Рисунок 2.4 – Інформаційні потоки функціоналу «Пошук вакансій за навичками» для Spring серверу

2.2 Типи користувачів

У системі буде присутній лише один тип користувача, який матиме можливість користуватися всім функціоналом. Вибір був зроблений на користь одного єдиного типу користувача, так як немає необхідності зберігати будь-які дані про користувача у віддаленій системі, вся необхідна інформація може зберігатися на пристрої.

2.3 Функціональні вимоги

F 1 Пошук вакансій за параметрами

FR 1.1 Треба вибрати хоча б один критерій пошуку.

FR 1.2 Додаткові параметри: ключові слова, досвід, локація, категорія.

FR 1.3 Параметри досвід, локація, категорії можна вводити вручну або вибрати зі списку доступних.

FR 1.4 Після введення всіх необхідних параметрів, повернеться список вакансій.

F 2 Пошук вакансій за навичками

FR 2.1 Треба ввести хоча б одну навичку.

FR 2.2 Після введення навичок, повернеться список вакансій.

F 3 Надання списку результатів

FR 3.1 Список повинен являти собою вакансії у вигляді «карток», у яких буде вся необхідна інформація у стислому виді.

FR 3.2 Можливість відкриття повного тексту вакансій.

FR 3.3 Можливість збереження вакансій.

F 3 Детальний перегляд вакансії

FR 3.1 Надання користувачу повної інформації про вакансію.

FR 3.2 Можливість зберегти вакансію.

FR 3.3 Можливість повернутись до списку вакансій.

F 4 Збереження вакансії

FR 4.1 Збереження вакансії до локальної бази даних.

F 5 Перегляд збережених вакансій

FR 5.1 Можливість перегляду збережених вакансій без доступу до інтернету.

FR 5.2 Можливість видалення вакансії із збережених.

2.4 Моделювання прецедентів (Use case View)

Для більш детального огляду системи побудуємо діаграми прецедентів (рис. 2.5 – 2.8)

Рисунок 2.5 – Діаграма прецедентів верхнього рівня

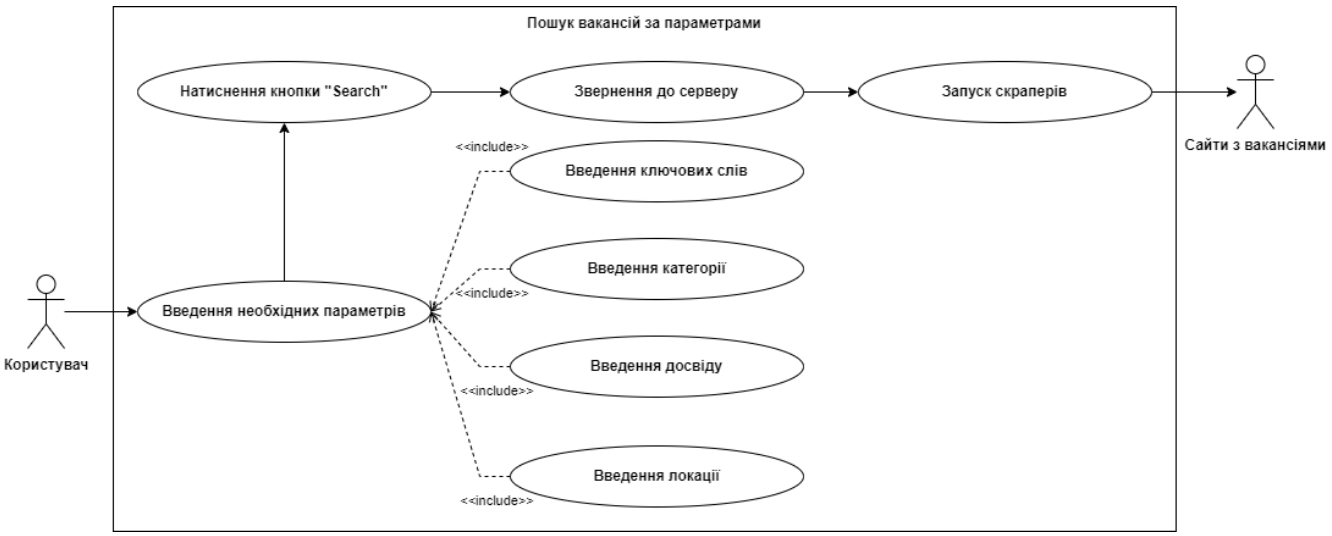


Рисунок 2.6 – Декомпозиція прецеденту «Пошук вакансій за параметрами»

Рисунок 2.7 – Декомпозиція прецеденту «Пошук вакансій за навичками»

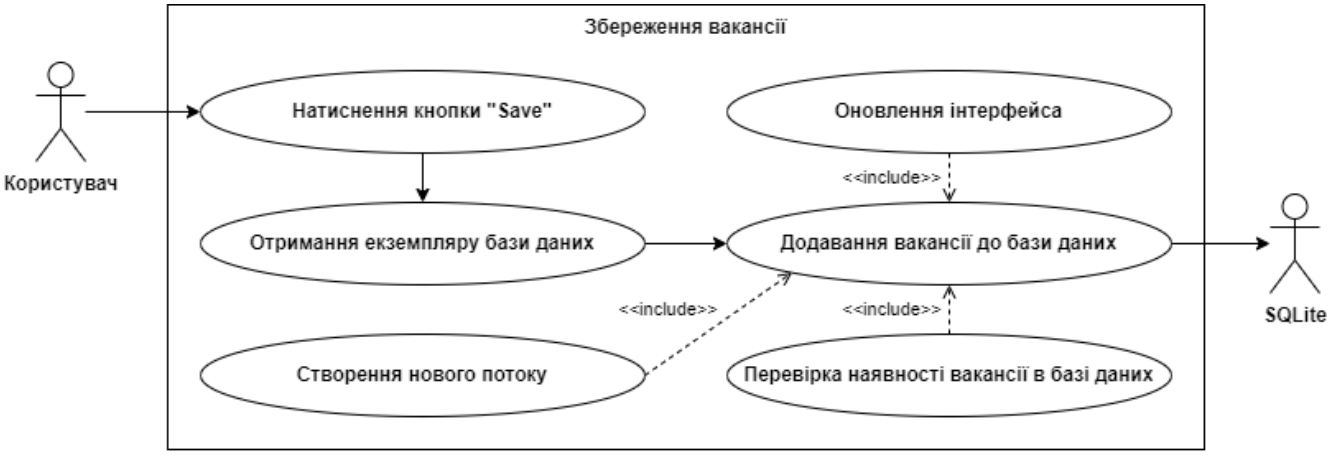


Рисунок 2.8 – Декомпозиція прецеденту «Збереження вакансії»

2.5 Нефункціональні вимоги

NF 1 Система являє собою Android – додаток.

NF 2 Застосунок підтримує Android – подібні ОС, з версією Android не нижче 5.0.

NF 3 Для пошуку вакансій потрібно підключення до Інтернету, для перегляду збережених – ні.

NF 4 Середній час відповіді 1-5 секунд.

NF 5 Інтерфейс повинен бути зручним, за останніми трендами та вимогами до інтерфейсу Android-застосунку.

2.6 Ідентифікація архетипу ІС

Інформаційна система буде мати клієнт-серверну архітектуру. Клієнт, Android-застосунок, реалізовано за архітектурним патерном MVVM. Сервер, який відповідає за обробку запитів клієнта, управління скраперами для пошуку вакансій, звернення до моделі для аналізу даних та бази даних, реалізовано за архітектурним патерном MVC та за допомогою фреймворку Spring.

2.7 Користувацький інтерфейс (UI View)

Початковий екран застосунку – екран введення параметрів пошуку. Користувачу потрібно заповнити хоча б один критерій пошуку.

Критерій «Key words» призначений для введення ключових слів, які будуть використовуватися для пошуку вакансій. Це просте поле для введення тексту.

Критерій «Experience» відповідає за робочий досвід шукача. На вибір надано наступні варіанти:

- No experience – відсутність досвіду роботу;
- 1 year – один рік досвіду або менше;
- 3 years – 3 роки досвіду;
- 5 years – 5 та більше років досвіду;

Критерій «Category» відповідає за категорію вакансій, яка цікавить користувача. Це поле для введення з автодоповненням. Потрібно ввести перші дві букви, та якщо буде знайдено збіг, застосунок запропонує відповідну категорію зі списку.

Критерій «Location» відповідає за локацію, яка цікавить користувача. Це поле для введення з автодоповненням. Присутні усі великі міста України та опції віддаленої роботи та релокації.

Для того щоб здійснити пошук необхідно натиснути кнопку «Search», яка відповідає за відправку запиту до серверу та навігацію до списку результатів.

Внизу екрану розташована панель навігації. За її допомогою користувач може переходити між екранами «Пошук вакансій за параметрами», «Пошук вакансій за навичками» та «Збереженні вакансії».

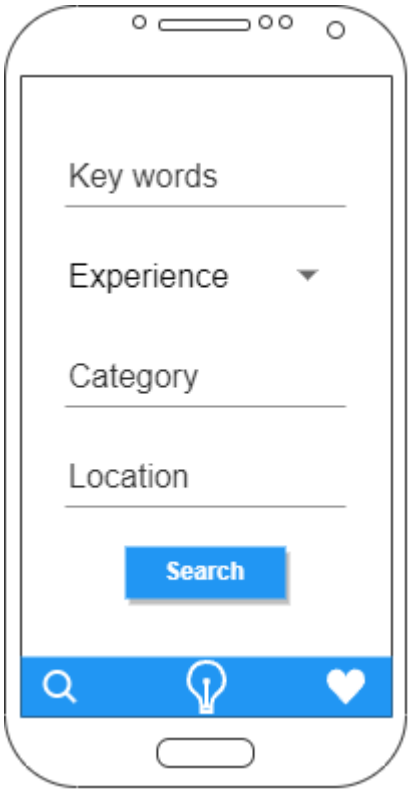


Рисунок 2.9 – Прототип екрану «Пошук вакансій за параметрами»

Екран «Пошук вакансій за навичками» містить поле з автодоповненням для введення навичок. Коли користувач починає вводити навичок, йому будуть запропоновані навички, які збігаються з введеним текстом. Введені навички, з’являються у вигляді кнопок під полем для введення. Якщо натиснути на них, вони будуть видалені. При закінченні введення навичок, необхідно натиснути на кнопку «Search», яка відправить запит до серверу та відкриє екран з результатами.

Рисунок 2.10 – Прототип екрану «Пошук вакансій за навичками»

Екран «Список вакансій» містить параметри або навички (відповідно до способу пошуку), список вакансій, які задовільнять пошуку та кнопку повернення до попереднього екрану. Вакансії у списку мають вигляд карток, у яких розташовані:

- заголовок вакансії;
- запропонована зарплата (якщо наявна);
- назва компанії, яка надає вакансію;
- локація;
- короткий опис вакансії;
- кнопка зберігання вакансії.

При натисненні на кнопку зберігання, вакансія буде збережена у локальній базі даних, та її функціонал і текст буде змінено на видалення вакансії з збережених. При натисненні на вакансію, відкриється детальний опис вакансії.

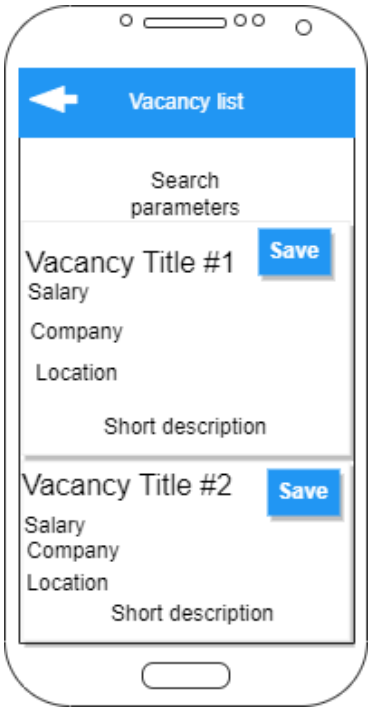


Рисунок 2.11 – Прототип екрану «Список вакансій»

Екран «Збережені вакансії» містить вакансії, збережені у локальній базі даних. Його функціонал повністю повторює функціонал екрану «Список вакансій», але є відміни у його дизайні та замість кнопки «Зберегти» присутня кнопка «Видалити», яка видаляє вакансію з локальної бази даних.

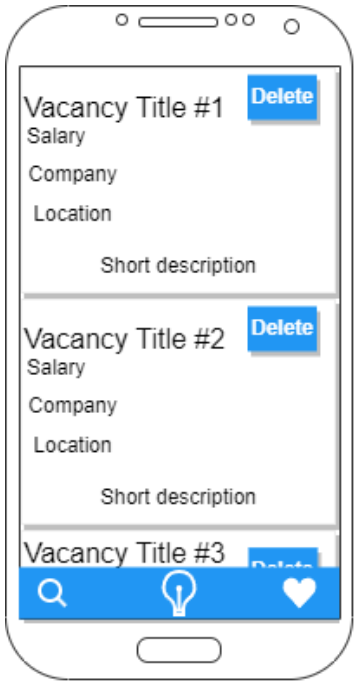


Рисунок 2.12 – Прототип екрану «Збережені вакансії»

Екран «Докладний огляд вакансії» містить повну інформацію про вакансію. До інформації, яка міститься на екрані «Список вакансій» додається повний опис та посилання на джерело, звідки ця вакансія була одержана.

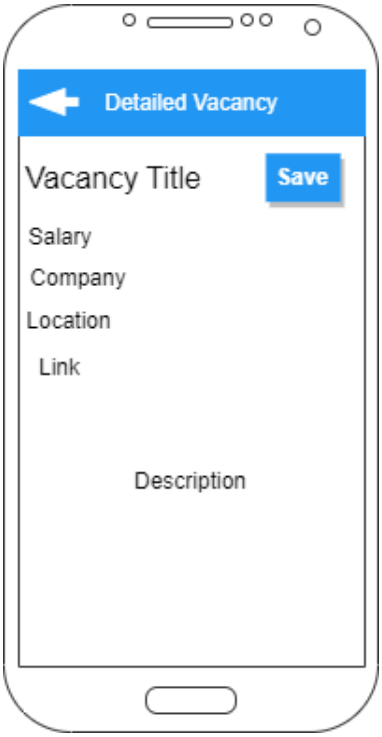


Рисунок 2.13 – Прототип екрану «Докладний огляд вакансії»

Побудуємо діаграму переходів між екранами (рис.2.14).

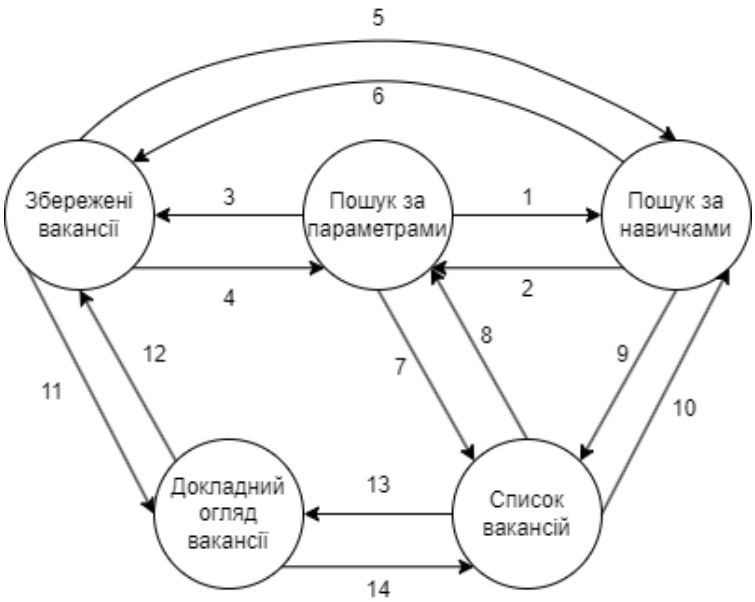


Рисунок 2.14 – Діаграма переходів

2.8 Логічне уявлення про ІС (Logical View)

Для наглядного уявлення системи, створимо діаграму логічного уявлення зв'язку компонентів системи. Користувач взаємодіє з UI частиною системи, яка обробляє його дії, отримує дані від нього, відсилає запити на серверну частину, обробляє відповіді від серверу та повертає представлення даних. В залежності від запиту, отриманого від UI частини, серверна частина або починає пошук вакансій на сайтах, або обробляє навички користувача та повертає дані з бази даних.

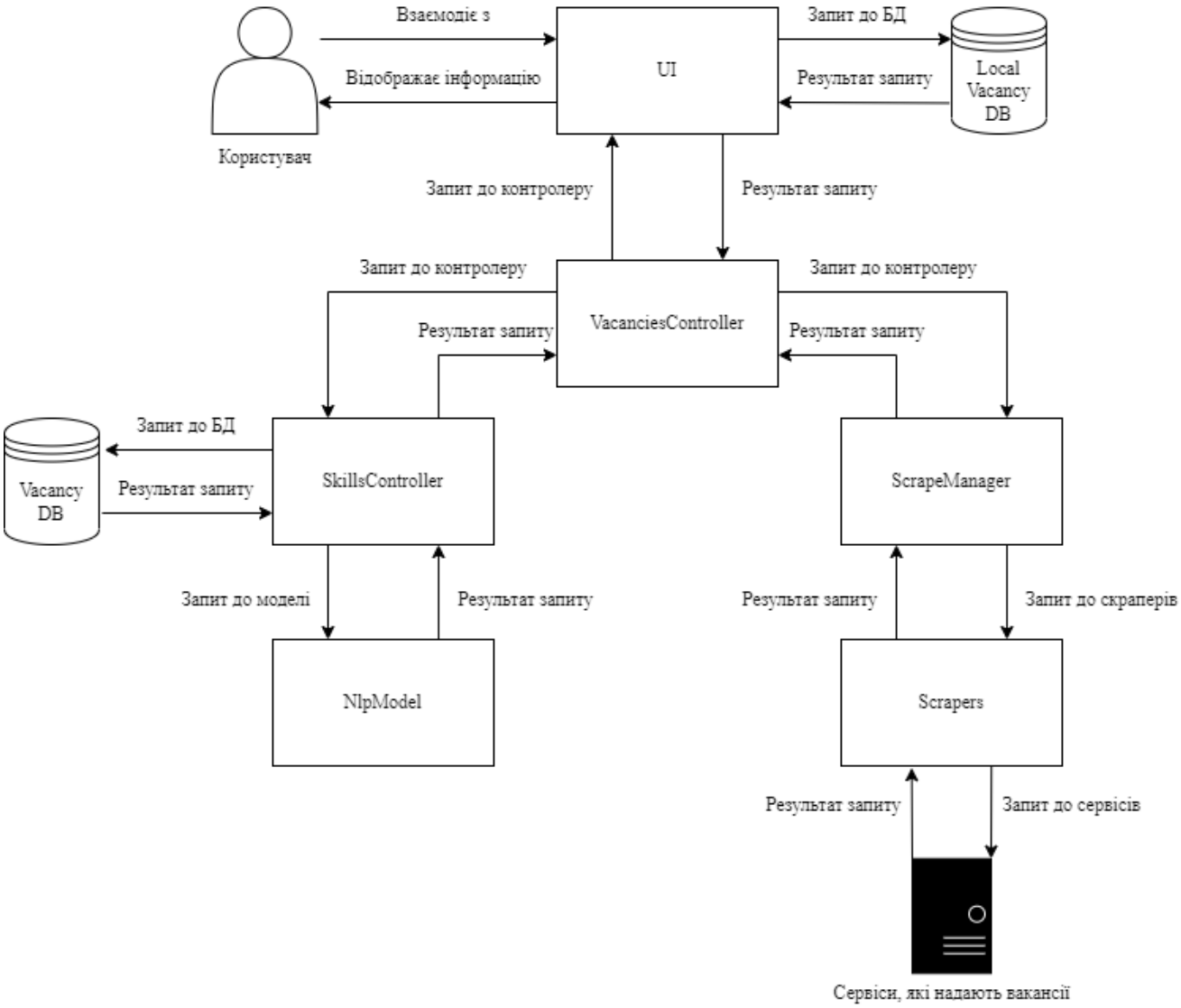


Рисунок 2.15 – Логічне уявлення ІС

2.9 Уявлення розгортання ІС (Deployment View)

Взаємодія користувача із системою відбувається через мобільний застосунок за допомогою фрагментів. Кожен екран має свій фрагмент, які обробляють дії користувача та повертають представлення даних. Роботою з даними займаються View-моделі. В них знаходиться логіка, яка відповідає за збереження даних та спілкування з сервером. Усі запити від клієнта спочатку приходять до контролера VacanciesController, який вирішує, що з ними робити. Якщо прийшов запит на пошук вакансій за параметрами, то він звертається до класу ScrapeManager, який відповідає за створення, запуск та передачу даних до скраперів, які шукають вакансії на сайтах. Якщо прийшов запит на пошук вакансій за навичками, то VacanciesController делегує це контролеру SkillsController, який спочатку звертається до моделі для аналізу даних, від якої отримує відповідні категорії вакансій, а потім звертається до бази даних, від якої отримує вакансії, за відповідними категоріями.

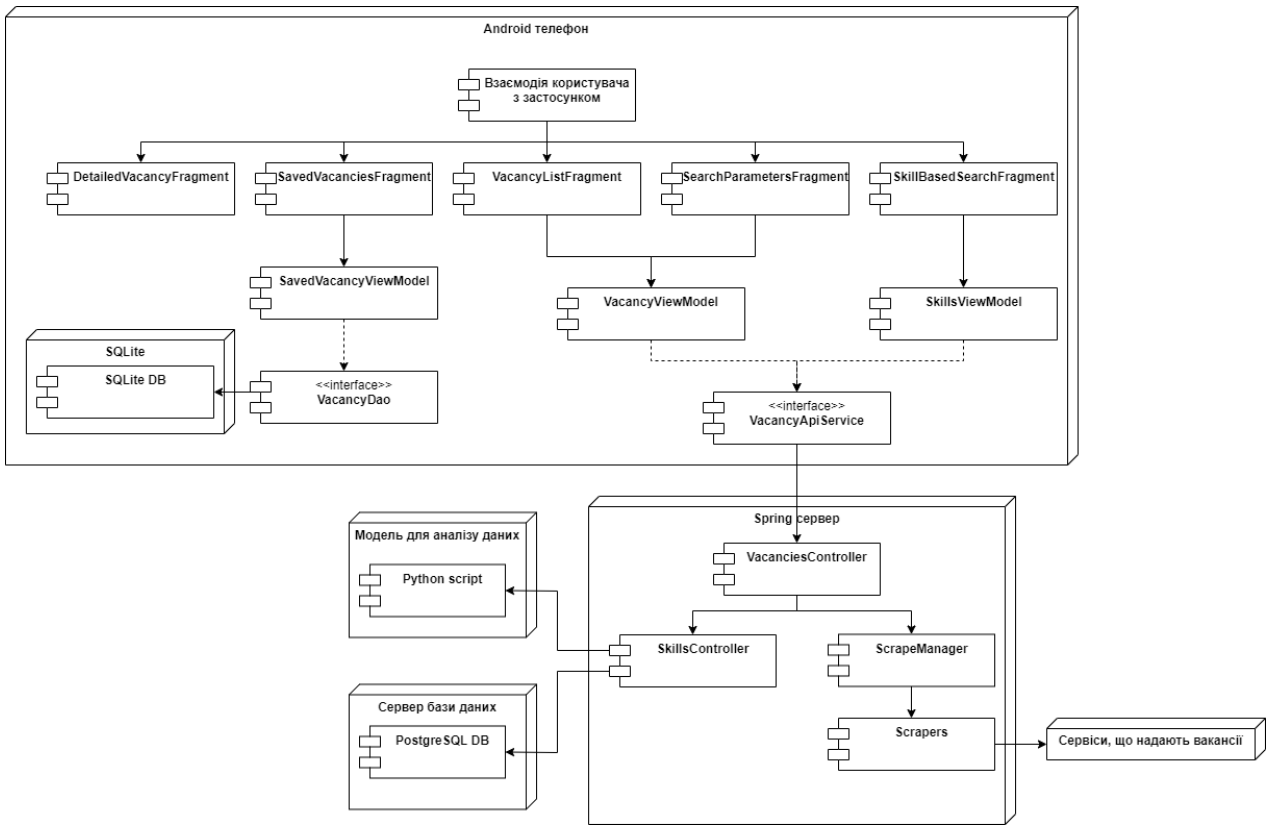


Рисунок 2.16 – Діаграма розгортання ІС

2.10 Уявлення шарів ІС (Design View)

Android-застосунок створений на підставі патерну MVVM, тому на діаграмі будуть присутні відповідні шари. View Layer являє собою шар, який містить фрагменти – представлення, завдяки яким користувач взаємодіє з застосунком. Вони обробляють дії користувача та отримують дані від нього. Натспуний шар – ViewModel Layer. Цей шар містить View-моделі, які відповідають за обробку даних користувача, та оновлюють моделі або передають їм дані на подальшу обробку. Model Layer – шар, у якому розташовані класи, які безпосередньо містять або оброблюють дані. Вони відповідають за зв’язок з сервером та базою даних. DB Layer – шар, який являє собою локальну базу даних.

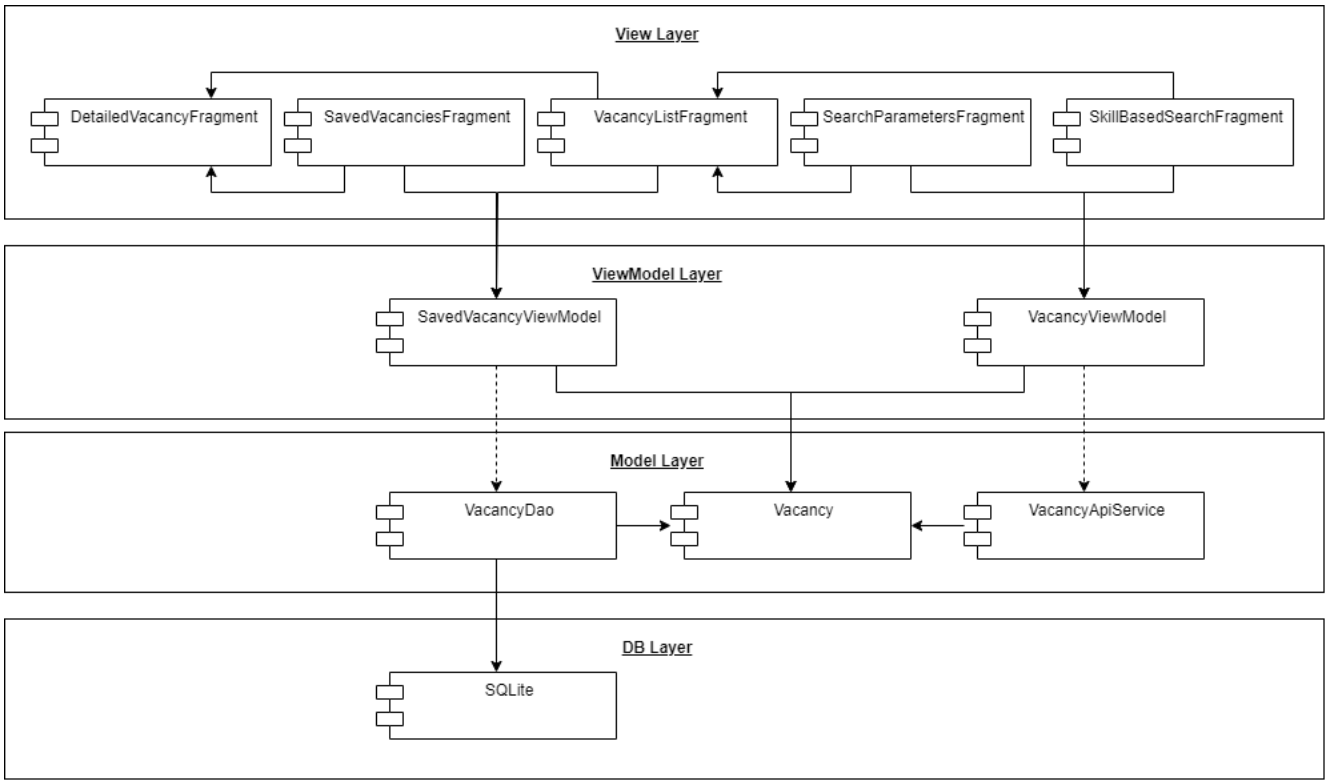


Рисунок 2.17 – Діаграма шарів Android-застосунку

На верхньому рівні діаграми шарів серверу – шар Controller Layer, який містить класи-контролери. Вони вирішують, що робити з даними, які надійшли та повертають оброблені дані клієнту. Головним контролером серверу є

VacancyController, який отримує дані від клієнта та на підставі, що це за дані, делегує подальшу обробку іншим класам. Наступний шар – Scrapers Layer. Це шар, який відповідає за роботу із скраперами. Якщо від клієнта прийшов запит на пошук вакансій за параметрами, то VacancyController звертається та передає дані до ScrapersManager, який створює та запускає скарпери, які шукають вакансій за критеріями на сайтах. Коли скрапери завершують свою роботу, вони повертають дані до ScrapersManager, який повертає дані до головного контролера VacancyController. Шар Model Layer містить класи даних, які використовуються класами з інших шарів. В ньому є сутність вакансії та складних параметрів пошуку. Шари DB Layer та NLP Layer містять базу даних та модель для обробки даних відповідно. Якщо до головного контролера прийшов запит на пошук вакансій за навичками, він передає управління контролеру SkillsController, який спочатку звертається до моделі для аналізу даних, яка повертає сумісні категорії вакансій з навичками, а потім звертається до бази даних, де вже ї отримує самі вакансії, які будуть повернути до користувача.

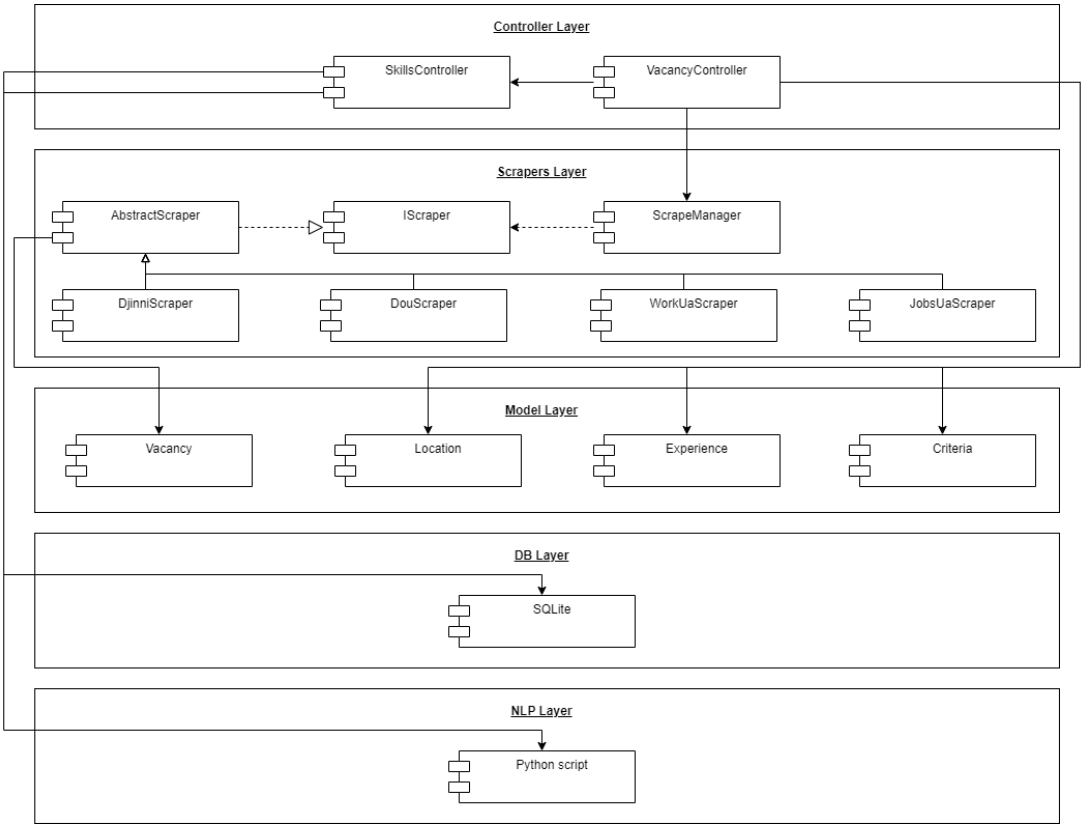


Рисунок 2.18– Діаграма шарів серверу

2.11 Уявлення процесів ІС (Process View)

Рисунок 2.19 – Діаграма послідовності для сценарію «Пошук вакансій за параметрами»

Рисунок 2.20 – Діаграма активності для сценарію «Пошук вакансій за параметрами»

Рисунок 2.21 – Діаграма послідовності для сценарію «Пошук вакансій за навичками»

Рисунок 2.22 – Діаграма активності для сценарію «Пошук вакансій за навичками»

Рисунок 2.23 – Діаграма послідовності для сценарію «Збереження вакансії»

Рисунок 2.24 – Діаграма активності для сценарію «Збереження вакансії»

2.12 Проектування бази даних

Єдине, що зберігається в системі – вакансії. На клієнті – вакансії, які зберігає користувач, на сервері – вакансії, які повертаються як відповідь на пошук вакансій за навичками.

vacancies	
id	integer
title	text
company	text
salary	text
location	text
link	text
description	text

Рисунок 2.25 – Таблиця vacancies у локальній базі даних

vacancies	
id	integer
title	text
company	text
salary	text
location	text
link	text
description	text
category	category

Рисунок 2.26 – Таблиця vacancies у базі даних на сервері

Поля таблиць:

– id – унікальний номер вакансії, за яким до них здійснюватиметься доступ;

- title – заголовок вакансії, що несе в собі інформацію про напрямок та позицію вакансії;
- company – назва компанії, яка надає вакансію;
- salary – зарплата, зазначена у вакансії. Вибрано тип Text, оскільки значення можуть бути різні: точне число в гривнях/доларах, діапазон значень, наприклад, $500-$1000, або текст, такий як «За результатами співбесіди»;
- location – місто, яке зазначене для вакансії, або віддалена робота.
- description – опис вакансії;
- category – категорія, за якою буде здійснюватися пошук за навичками.

2.13 Уявлення інтерфейсів ІС (UI View)

Основою Android-застосунку є клас MainActivity, який є точкою входу. Він містить контейнер фрагментів, у який додаються необхідні, у міру дій користувача. Усі екрани застосунку мають три елементи – xml-файл розмітки, у якому формується інтерфейс за допомогою View-об’єктів, Binding, який є представленням xml-розмітки у вигляді класу, який зберігає View-об’єкти, що спрощує доступ до них, та класів Fragment, які містять основну логіку взаємодії з користувачем. Fragment-класи відповідають за передачу даних від користувача до View-моделей, вирішують, що робити з натиснутою кнопкою та виконують навігацію між екранами. Як раніше було зазначено, Fragment-класи не виконують обробку даних та не взаємодіють з сервером або базою даних, для цього існують інші класи – ViewModel та Model-класи. ViewModel-класи є зв’язком між представленням даних, та класами, які їх оброблять та зберігають. Model-класи або являють собою якісь дані, або містять бізнес-логіку системи, яка обробляє дані. За допомогою Model-класів у застосунку здійснюється взаємодія з базою даних, а саме читання та запис даних, та взаємодія з сервером, до якого дані відправляються на подальшу обробку. Сервер, отримавши дані, вирішує, що з ними робити, обробляє їх відповідним чином та повертає дані до моделі. Після того, як дані у моделі

оновилися, Fragment-класи, які спостерігають за даними ViewModel, оновлюють користувацький інтерфейс відповідним чином.

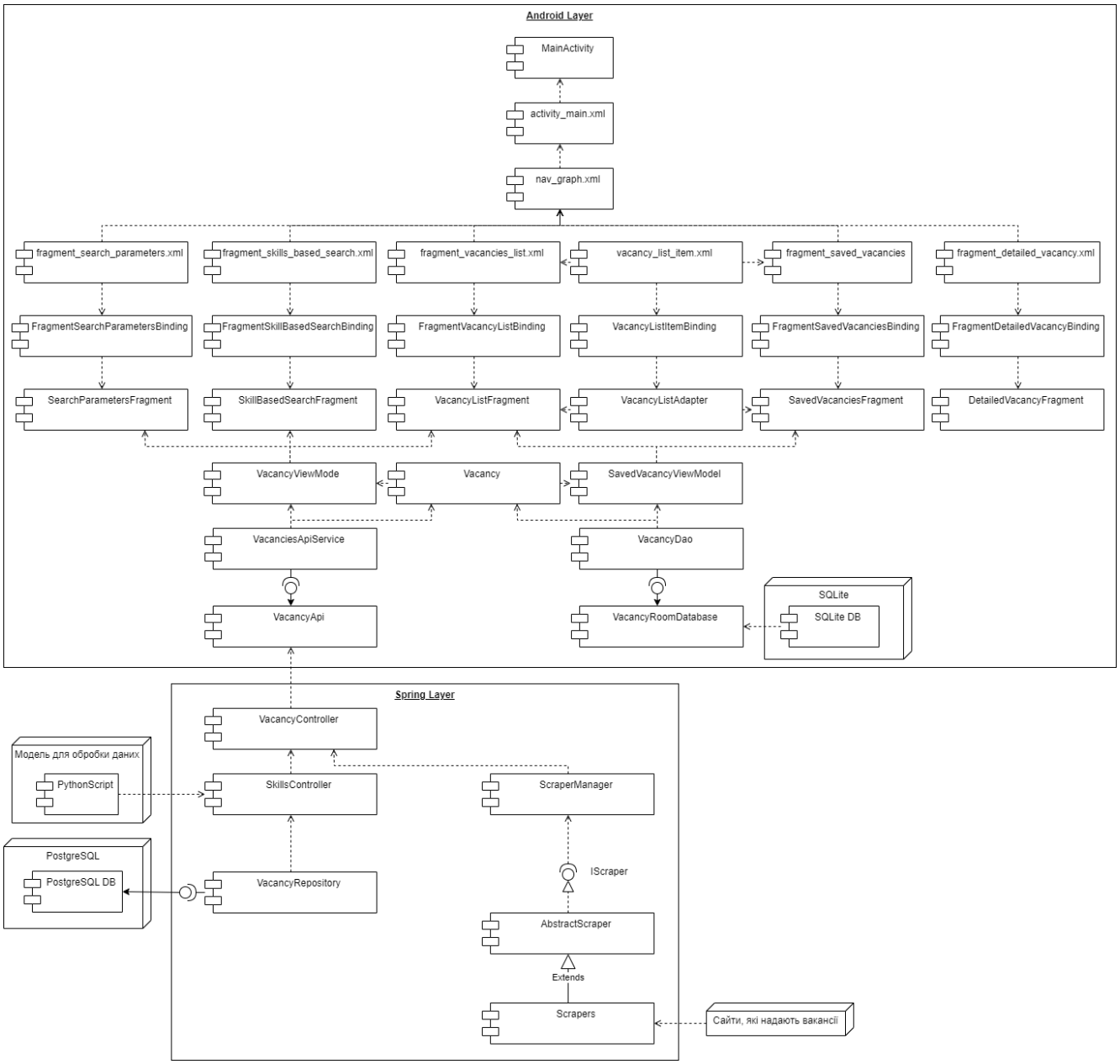


Рисунок 2.27 – Діаграма уявлення інтерфейсів ІС

2.14 Уявлення безпеки ІС

Система не зберігає та не обробляє особисті дані користувача чи якісь інші дані, які потрібно захищати.

2.15 Опис стеку технологій

Мови програмування, фреймворки та бібліотеки, які були використані в процесі розробки:

– мова Kotlin 1.7.10 – мова на якій була написана Android частина застосунку;

– JUnit 4.13.2 – фреймворк для створення тестів;

– бібліотека Kotlinx 1.10.1 – потрібна для роботи з корутинами;

– бібліотека Retrofit 2.9.0 – допомагає здійснювати спілкування з RESTful серверами;

– бібліотека Moshi 1.13.0 – перетворює об’єкти JSON у Java або Kotlin класи;

– бібліотека Room 2.5.1 – надає абстракцію над базою даних, що облегшує взаємодію з базою даних;

– Spring 5.3.4 - фреймворк, на якому написано Spring сервер.

Застосунки, які були використані для написання програмного коду:

– Android Studio 2022.1.1 – IDE, в якій здійснювалася розробка Android частини;

– IntelliJ IDEA Community Edition 2022.3.2 – IDE, в якій здійснювалася розробка Spring серверу.

2.16 Висновок до другого розділу

В даному розділі були зазначені мета та задачі інформаційної системи, що розробляється. Були визначені типи користувачів у системі, зазначені функціональні та нефункціональні вимоги до системи. Були змодельовані прецеденти усіє системи, та декомпозовані основні з них. Були наведені прототипи користувальницького інтерфейсу з їх докладним описом. Були створені діаграми, які допомагають зрозуміти роботу та архітектуру системи, від загальних, таких як діаграми логічного уявлення та розгортання, до специфічних, як діаграма уявлення

інтерфейсів. Була спроектована база даних та стек технологій, з поясненням, щодо використаних бібліотек та фреймворків.

3 РЕАЛІЗАЦІЯ ЗАСТОСУНКУ ЗБОРУ ТА АГРЕГАЦІЇ ДАНИХ ПРО ВАКАНСІЇ

ТРЕБА ЗРОБИТИ

3.1 Уявлення про структуру проекту ІС

3.2 Уявлення про структуру класів

3.3 Інфраструктурне уявлення ІС (Infrastructure View)

3.4 Стратегія доставки продукту (Delivery Strategy View)

3.5 Управління програмним кодом системи

3.6 Розрахунок метрики програмного коду системи

3.7 Контрольний список по якості реалізації ІС

3.8 Тестування інформаційної системи

3.8.1 Розробка тест-кейсів для функціонального тестування системи

3.8.2 Протокол проведення функціонального тестування системи (Functional acceptance testing)

3.9 Інструкція користувача

3.10 Висновки до третього розділу

ВИСНОВКИ

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДЖЕРЕЛА ПОКИ НЕ ОФОРМЛЮВАВ

1.https://www.jetbrains.com/products/compare/?product=idea&product=idea-ce

2.https://developer.android.com/codelabs/basic-android-kotlin-training-getting-data-internet?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-kotlin-unit-4-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-training-getting-data-internet#4

3. https://github.com/square/moshi

4. https://pandas.pydata.org/docs/getting_started/overview.html

5. https://github.com/google/gson

6. https://en.wikipedia.org/wiki/Spring_Framework

7. https://www.geeksforgeeks.org/mvvm-model-view-viewmodel-architecture-pattern-in-android/

8. https://refactoring.guru/design-patterns/observer

9. https://www.geeksforgeeks.org/mvc-design-pattern/

ДОДАТОК А ЛІСТИНГ ПРОГРАМНОГО КОДУ

ДОДАТОК Б МОКАПИ